Contrôle décentralisé d'un essaim de robots par réseaux de neurones cellulaires

CeNN-Inspired Swarm Robotics

18 octobre 2026

Résumé

Ce mémoire étudie le contrôle décentralisé d'un essaim de robots simulés par un réseau de neurones sur graphes partagé. Chaque robot ne traite que son état local, les messages de ses voisins proches et une consigne commune de forme. L'objectif est de former des nuages de points représentant des formes géométriques et symboliques, tout en examinant le comportement face aux pannes et aux changements de taille de l'essaim. La contribution principale est une chaîne expérimentale reproductible : simulateur 2D, jeu de formes, contrôleur GNN, entraînement différentiable et protocole d'évaluation. Un premier protocole pilote montre que le contrôleur appris obtient une meilleure adéquation à une cible circulaire qu'une règle locale de regroupement. Cette observation reste exploratoire : la baseline ne reçoit pas la même information de consigne et l'étude finale devra augmenter le nombre de répétitions et inclure des comparaisons plus équitables.
Mots-clés : robotique en essaim, contrôle décentralisé, réseaux de neurones sur graphes, formation de formes, apprentissage différentiable.

Abstract

This thesis studies decentralized control of a simulated robot swarm with a shared graph neural network. Each robot only processes its local state, messages from nearby neighbours, and a common shape instruction. The main contribution is a reproducible experimental pipeline combining a two-dimensional simulator, a shape dataset, a local GNN controller, differentiable training, and an evaluation protocol. Preliminary experiments show that the learned controller is closer to a circular target than a local gathering rule. This observation remains exploratory because the baseline does not receive the same target instruction and the final study must use more repetitions and information-equivalent comparisons.

Keywords : swarm robotics, decentralized control, graph neural networks, shape formation, differentiable learning.

Table des matières

Résumé **1**

Abstract **2**

1 Introduction **10**

1.1 Contexte 10
1.2 Problématique et question de recherche 11
1.3 Objectifs et contributions 12
1.4 Hypothèses et périmètre 12
1.5 Choix du problème étudié 13
1.6 Critères de réussite 14
1.7 Démarche générale du mémoire 14
1.8 Organisation du mémoire 15

2 État de l'art **16**

2.1 Robotique en essaim et contrôle décentralisé 16
2.2 Inspirations biologiques et exigences d'ingénierie 17
2.3 Consensus, agrégation et convergence distribuée 18
2.4 Flocking et comportements locaux 19
2.5 Formation de formes : de la géométrie imposée à la consigne partagée . . . 20
2.6 Réseaux de neurones sur graphes et passage de messages 21
2.7 Variants des GNN et justification de l'agrégation 23
2.8 Induction, partage des paramètres et changement d'échelle 23
2.9 Apprentissage de contrôleurs décentralisés par GNN 24
2.10 Apprentissage de contrôle : imitation, simulation différentiable, multi-agents 25
2.11 Lecture critique des comparaisons 26
2.12 De la règle écrite à la règle apprise 26
2.13 Consigne, perception et déploiement 27
2.14 Synthèse comparative et positionnement 27
2.15 Réseaux cellulaires et positionnement du mémoire 29

3 Méthode et architecture logicielle **30**
3.1 Formulation du problème . . . 30
3.2 Simulateur et hypothèses . . . 31
3.3 Formes cibles . . . 31
3.4 Génération des nuages de points cibles . . . 32
3.5 Contrôleur GNN . . . 34
3.6 Apprentissage . . . 34
3.7 Algorithme d'entraînement . . . 37
3.8 Reproductibilité et baselines . . . 37
3.9 Organisation de l'implémentation . . . 38
3.10 Restitution des trajectoires . . . 39
3.11 Informations locales et information de mission . . . 39
3.12 Configurations d'entraînement . . . 39
3.13 Mesures utilisées pendant et après l'apprentissage . . . 40
3.14 Définition opérationnelle des mesures . . . 41
3.15 Déroulement détaillé d'un épisode . . . 41
3.16 Choix des hyperparamètres . . . 42
3.17 Valeurs numériques retenues . . . 43
3.18 Gestion des formes et du curriculum . . . 44
3.19 Architecture logicielle et reproductibilité . . . 44
3.19.1 Vue d'ensemble des modules . . . 45
3.19.2 Structure du dépôt . . . 45
3.19.3 Cycle d'exécution d'un épisode . . . 46
3.19.4 Configuration et traçabilité . . . 47
3.19.5 Tests et contrôles de non-régression . . . 48
3.19.6 Exposition locale de la démonstration . . . 49
3.19.7 Coût de calcul et choix de mise en œuvre . . . 49
3.19.8 Visualisation et inspection . . . 50

4 Expériences et résultats **51**
4.1 Protocole pilote . . . 51
4.2 Résultats préliminaires . . . 52
4.3 Effet du nombre de robots . . . 53
4.4 Lecture des scénarios de panne . . . 56
4.5 Dispersion des épisodes et atteinte du seuil . . . 57
4.6 Effet des retraits sur les mesures . . . 58
4.7 Curriculum de formes . . . 60
4.8 Exemples, résultats négatifs et limites de lecture . . . 62
4.9 Discussion des résultats . . . 63

4.10 Lien entre agrégation et résultats observés 64
4.11 Analyse des métriques 64
4.12 Menaces à la validité 65
4.13 Plan d'expériences pour la campagne finale 66
4.14 Interprétation prudente des résultats pilotes 66

5 Discussion et conclusion **68**
5.1 Interprétation des résultats 68
5.2 Comparaison avec les travaux antérieurs 69
5.3 Menaces à la validité 69
5.3.1 Validité interne 69
5.3.2 Validité externe 70
5.3.3 Validité de construit 70
5.4 Implications pour la conception de contrôleurs décentralisés 71
5.5 Considérations pratiques et sociales 71
5.6 Suite expérimentale 72
5.7 Conclusion et perspectives 73

Table des figures

1.1 Illustration conceptuelle d'une formation de forme : à gauche, des départs aléatoires ; au centre, une étape intermédiaire ; à droite, une formation approchée du contour cible. L'affectation représentée illustre le scénario général et ne constitue pas un résultat du contrôleur appris. 13
1.2 Vue d'ensemble de la chaîne expérimentale. Une perte globale est utilisée pendant l'apprentissage ; lors de l'exécution, chaque robot applique une politique partagée à partir de son observation locale, de ses voisins et de la consigne de mission. 15

2.1 Consensus sur un graphe linéaire de cinq agents. À gauche, évolution des valeurs à chaque passe ; à droite, décroissance de l'écart entre les valeurs extrêmes sur une échelle logarithmique. 19
2.2 Les trois contributions d'une règle de flocking appliquées au robot orange : attraction vers le centre de masse des voisins (cohésion), éloignement du voisin le plus proche (séparation) et alignement sur la direction moyenne du groupe. 20
2.3 Principe d'agrégation employé dans le contrôleur. Les messages individuels sont calculés pour chaque voisin, puis leur moyenne est utilisée pour mettre à jour l'état local du robot. 22
2.4 Portée de l'information selon le nombre de tours de messages. Chaque anneau correspond à un saut supplémentaire dans le graphe de communication ; les expériences du mémoire utilisent deux tours. 22

3.1 Schéma d'un essaim. Le robot orange ne peut échanger qu'avec les robots situés dans sa portée de communication ; les robots distants ne sont pas accessibles directement. 32
3.2 Chaîne de préparation d'une cible : image rendue, masque binarisé, puis nuage de points échantillonné et normalisé. L'exemple provient du jeu de formes standard. 33
3.3 Calcul local réalisé pour un robot lors d'un tour de passage de messages. Les mêmes blocs de calcul et les mêmes paramètres sont partagés dans l'essaim. 34

3.4 Lecture géométrique de la distance de Chamfer symétrique : en pointillés, les appariements vers le voisin le plus proche, pris dans les deux sens entre les robots P et les points cibles Q. 35
3.5 Composition de la fonction de perte. Les quatre termes, pondérés par les coefficients λ versionnés dans les configurations, sont sommés en une seule perte d'épisode rétropropagée vers les paramètres partagés. 36
3.6 Les quatre étapes du curriculum réduit, avec un exemple de cible et le nombre de formes ajoutées à chaque étape. Les cibles des étapes antérieures sont conservées dans le lot d'entraînement. 36
3.7 Rétropropagation à travers les pas de simulation. Les T pas forment un graphe de calcul enregistré ; le gradient de la perte finale remonte jusqu'aux paramètres partagés du contrôleur à travers toute la trajectoire. 37
3.8 Organisation en modules du prototype. Chaque module consomme et produit des artefacts explicites ; la séparation entre simulation, modèle et mesure permet d'évaluer plusieurs contrôleurs sur les mêmes conditions initiales. 45
3.9 Cycle de vie d'un épisode. La boucle centrale est répétée T fois ; la mesure finale ne porte que sur les robots actifs et alimente la perte (entraînement) ou les métriques (évaluation). 47

4.1 Évolution de la perte pendant l'entraînement initial du cercle, pour la graine 41. Cette courbe sert de diagnostic d'optimisation et non d'intervalle de confiance. 52
4.2 Synthèse du protocole pilote calculée à partir de 240 épisodes. À gauche, l'effet de la taille de l'essaim sans panne ; à droite, l'effet des pannes avec 20 robots. Une valeur plus faible de distance de Chamfer indique une meilleure adéquation à la cible circulaire. 53
4.3 Configuration finale obtenue pour l'entraînement du cercle. Les robots représentent l'approximation discrète de la cible par l'essaim. 54
4.4 Distance moyenne de Chamfer (moyenne $\pm$ 1 écart-type sur dix épisodes) selon la fraction de robots retirés à mi-épisode, pour 20 robots : à gauche, le GNN ; à droite, la règle de regroupement. Le seuil de succès $\tau = 0{,}5$ est indiqué pour le GNN. 55
4.5 Chronologie d'un épisode avec retrait à mi-parcours. Les robots retirés (gris) sont figés et exclus du graphe de communication ; la mesure finale porte sur les robots actifs restants. 56
4.6 Distance moyenne de Chamfer lors du passage à l'échelle dans le protocole pilote. Cette figure doit être complétée par les valeurs numériques et leur dispersion dans la campagne finale. 57

4.7 Schéma de la mesure de convergence : la distance de forme (bleue) est comparée au seuil τ (orange) à chaque pas ; le premier franchissement définit le pas de convergence t_{conv}. La courbe est illustrative et n'est pas extraite des épisodes du pilote. 59
4.8 Distance de Chamfer finale des dix graines du pilote à 20 robots sans retrait, triées par distance croissante. La ligne verte marque le seuil de succès $\tau = 0{,}5$ et la ligne pointillée la moyenne ; le coloris indique le franchissement du seuil rapporté au tableau 4.3. 59
4.9 Lecture visuelle du tableau 4.5 : pertes moyennes à la fin de chaque étape du curriculum, séparées entre cibles déjà vues et cibles introduites par l'étape, avec le nombre de cibles de chaque étape. 61
4.10 Pertes enregistrées pendant les quatre étapes du curriculum réduit. Chaque graphe correspond à une étape ; l'axe des ordonnées est commun, ce qui permet de comparer l'amplitude des variations entre les étapes. 61
4.11 Planche de visualisation issue de la dernière étape du curriculum réduit. Elle illustre des formations approximatives et ne constitue pas encore une validation de lisibilité des symboles. 62

5.1 Contrôleurs comparés ou prévus : présence des messages de voisins, de la consigne de forme et de paramètres appris, ainsi que le statut de chaque comparaison. Seules les deux premières lignes ont été évaluées dans le pilote ; les autres constituent les comparaisons à information équivalente à réaliser. 72

Liste des tableaux

2.1 Comparaison des grandes familles d'approches pour la formation de formes décentralisée. 28

3.1 Principaux symboles utilisés pour décrire l'essaim, la politique et l'évaluation. 30
3.2 Hyperparamètres des trois campagnes d'entraînement, repris des fichiers de configuration versionnés. 43
3.3 Modules du prototype, responsabilités et artefacts produits. 46
3.4 Fichiers de configuration du projet et objet de chacun. 47
3.5 Propriétés vérifiées par la suite de tests unitaires, par composant. 48
3.6 Routes de l'API locale de visualisation. 49

4.1 Résultats du protocole pilote à 20 robots (10 graines par condition). 52
4.2 Passage à l'échelle sans panne dans le protocole pilote, avec dix épisodes par condition. 55
4.3 Épisodes du pilote à 20 robots sans retrait, contrôleur GNN, triés par distance de forme. Le taux de collisions est normalisé par le nombre de paires et la durée de l'épisode. 57
4.4 Distance de forme moyenne et écart-type (entre parenthèses) pour 20 robots selon la fraction de robots retirés à mi-épisode. 58
4.5 Perte par cible à la fin de chaque étape du curriculum réduit, séparée entre cibles déjà vues et cibles introduites par l'étape. 60

Chapitre 1

Introduction

1.1 Contexte

La robotique en essaim étudie la coopération d'un grand nombre d'agents relativement simples. Plutôt que de confier la mission à un robot chef, elle cherche à faire émerger un comportement collectif à partir de décisions locales, de communications limitées et de règles partagées [3]. Cette organisation présente plusieurs intérêts : elle répartit la charge de calcul, évite un point unique de défaillance et peut, en principe, tolérer l'ajout ou la perte de quelques unités. Elle est pertinente pour des tâches telles que l'exploration, la surveillance distribuée, le transport coopératif ou l'occupation d'une zone.

Ce potentiel s'accompagne toutefois d'une difficulté fondamentale. Le résultat recherché est global — par exemple couvrir une région, maintenir une formation ou dessiner une figure — alors que chaque robot ne dispose que d'une perception partielle de la situation. Les règles classiques de comportement collectif, inspirées notamment du flocking, combinent cohésion, séparation et alignement pour produire des mouvements de groupe plausibles [26, 21]. Elles restent précieuses par leur simplicité et leur interprétabilité, mais leur conception devient délicate lorsque la tâche exige une géométrie précise ou un ensemble varié de cibles.

La formation de formes constitue un cas d'étude particulièrement révélateur. Rassembler des robots vers une même zone ne garantit ni une couverture homogène, ni l'apparition d'un cercle, d'une lettre ou d'un symbole. Pour atteindre une forme, le contrôleur doit simultanément guider les robots vers le contour demandé, éviter les rapprochements excessifs et conserver une distribution suffisante sur l'ensemble de la cible. Des contrôleurs décentralisés spécialisés ont été proposés pour la génération de courbes ou de formations progressives [12, 18]. Leur existence montre la faisabilité du contrôle local, mais aussi la nécessité de définir soigneusement les informations disponibles et les mécanismes de décision.

Dans ce mémoire, la règle locale n'est pas entièrement écrite à la main. Elle est repré-

sentée par un réseau de neurones sur graphes (*Graph Neural Network*, GNN). Le graphe encode naturellement un essaim : les robots sont les nœuds et les liens de communication entre voisins sont les arêtes. Une même politique paramétrée est partagée par tous les robots ; elle agrège les messages provenant du voisinage puis produit une commande de vitesse. Les GNN constituent ainsi une piste adaptée pour apprendre une loi locale tout en conservant une structure d'exécution distribuée [9, 2, 31].

1.2 Problématique et question de recherche

Le contrôle centralisé fournit une solution conceptuellement simple à la formation de formes : un planificateur connaît les positions de tous les robots, associe éventuellement chaque robot à un point de la cible, puis transmet les commandes. Cette hypothèse est peu compatible avec un essaim dont les échanges sont limités par la portée radio, dont la taille varie et dont certains agents peuvent disparaître. Elle masque également la question principale de la robotique en essaim : comment un résultat collectif peut-il être obtenu lorsqu'aucun agent ne possède directement l'état complet du groupe ?

Le présent travail étudie donc la question suivante :

> *Un réseau de neurones sur graphes, exécuté à partir d'informations locales et d'une consigne commune de forme, peut-il apprendre à former des configurations spatiales et conserver un comportement utile en présence de pannes ou d'un changement de taille de l'essaim ?*

Cette formulation appelle trois précisions. Premièrement, le caractère décentralisé concerne la *décision de mouvement* : chaque robot applique le même contrôleur et n'échange qu'avec ses voisins à portée. Deuxièmement, la forme à atteindre est une consigne de mission partagée avant l'exécution ; elle n'est pas découverte spontanément par l'essaim. Selon la configuration expérimentale, cette consigne prend la forme d'un identifiant de cible ou d'une information géométrique locale relative au contour. Enfin, l'entraînement est centralisé dans un simulateur différentiable : une perte globale peut y être calculée pour ajuster les poids, alors que la politique déployée ne reçoit pas les positions des robots distants.

La question générale est déclinée en quatre sous-questions :

1. Le contrôleur appris réduit-il l'écart entre la configuration finale et une forme cible à partir de positions initiales aléatoires ?
2. L'échange de messages entre voisins apporte-t-il un bénéfice par rapport à des règles locales ou à une politique sans communication, à information de consigne comparable ?
3. La politique à paramètres partagés reste-t-elle applicable lorsque le nombre de robots et le nombre de voisins changent ?

4. Comment la qualité de la formation se dégrade-t-elle lorsque des robots sont retirés pendant un épisode ?

1.3 Objectifs et contributions

L'ambition du mémoire n'est pas de valider immédiatement une flotte de robots physiques. Il s'agit d'abord de construire un banc d'essai cohérent et reproductible permettant d'étudier une hypothèse de contrôle par GNN. Les principales contributions du travail sont les suivantes :

— un simulateur bidimensionnel qui représente la position, la vitesse, le voisinage de communication, les limites de l'arène et la perte de robots ;
— un jeu de formes converties en nuages de points normalisés, comprenant des géométries, des chiffres, des lettres et des nombres ;
— un contrôleur GNN à passage de messages, partagé entre tous les robots et compatible avec un voisinage de taille variable ;
— une procédure d'apprentissage direct à travers le simulateur, complétée par un curriculum allant de formes simples vers des formes symboliques ;
— un protocole d'évaluation qui mesure la proximité à la cible, les collisions, les pannes et le passage à l'échelle, tout en séparant les résultats pilotes des conclusions définitives.

Le titre déposé emploie l'expression « réseaux de neurones cellulaires ». L'analogie est justifiée par le fait que tous les robots appliquent la même règle et n'interagissent qu'avec un voisinage limité, à la manière d'unités cellulaires. L'implémentation réalisée est toutefois un GNN à passage de messages sur un graphe dynamique. Dans la suite du document, le terme GNN est donc utilisé pour décrire précisément l'architecture ; toute modification officielle de l'intitulé reste soumise à l'accord de l'encadrement.

1.4 Hypothèses et périmètre

Les expériences sont réalisées dans une arène plane sans obstacle fixe. Chaque robot possède une position et une vitesse en deux dimensions ; les déplacements sont représentés par une dynamique simplifiée à vitesse directement commandée. Deux robots peuvent communiquer lorsque leur distance est inférieure à une portée définie. Les mesures et les messages sont supposés exacts, instantanés et sans perte. Un robot déclaré en panne est retiré de la dynamique active et du graphe de communication.

Ces choix permettent d'isoler l'effet de l'architecture de contrôle et de l'apprentissage. Ils excluent volontairement plusieurs problèmes importants : estimation de pose distribuée, incertitude des capteurs, délais de communication, obstacles, dynamique réaliste

des actionneurs, consommation énergétique et validation sur matériel. Le projet ne prétend donc pas démontrer un transfert direct vers des robots réels. Ces limites constituent au contraire des axes de discussion et de prolongement.

1.5 Choix du problème étudié

Le choix de la formation de formes répond à une raison méthodologique. Il est suffisamment simple pour être représenté dans un simulateur 2D et visualisé directement, mais il reste plus exigeant qu'un simple rassemblement. Dans un rassemblement, plusieurs robots peuvent finir dans une même zone tout en donnant l'impression que la tâche est accomplie. Dans une formation, ce comportement est insuffisant : les robots doivent aussi se répartir sur les différentes parties de la cible. Cette exigence rend visibles plusieurs difficultés de contrôle distribué, notamment la couverture, les collisions et les ruptures de connectivité. La figure 1.1 illustre cette progression, du départ aléatoire à la formation approchée du contour.

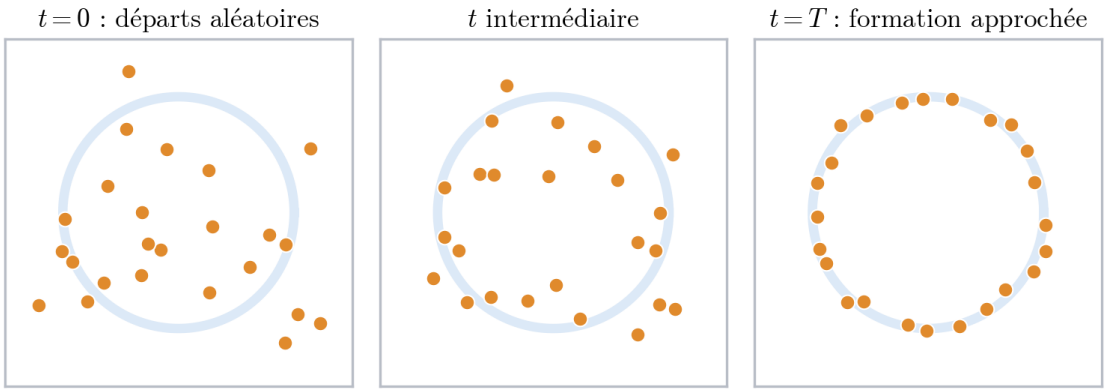


FIGURE 1.1 – Illustration conceptuelle d'une formation de forme : à gauche, des départs aléatoires ; au centre, une étape intermédiaire ; à droite, une formation approchée du contour cible. L'affectation représentée illustre le scénario général et ne constitue pas un résultat du contrôleur appris.

Le cercle sert de premier cas de validation. Sa géométrie est régulière et son contour peut être échantillonné facilement. Les chiffres, les lettres et les nombres permettent ensuite de vérifier que le pipeline ne dépend pas exclusivement d'une forme convexe et symétrique. Leur intérêt n'est pas de faire de la reconnaissance de caractères. Ils fournissent plutôt des contours présentant des segments, des angles, des régions moins régulières et parfois plusieurs parties visuellement distinctes. Une lettre formée de manière approximative reste donc un résultat de contrôle spatial, non la preuve qu'un système comprend un symbole.

Le projet adopte par ailleurs une vision pragmatique de la décentralisation. Il serait trompeur d'affirmer qu'un robot ne reçoit aucune information commune : toute mission suppose au minimum un objectif partagé. Ici, la forme à réaliser est connue avant le début de l'épisode. En revanche, la décision instantanée de mouvement ne dépend pas d'un tableau central contenant toutes les positions de l'essaim. Cette distinction entre une consigne commune et une observation globale est importante pour interpréter les résultats.

1.6 Critères de réussite

Une expérience ne sera pas considérée comme réussie sur la seule base d'une animation convaincante. La première condition est que les robots se rapprochent du nuage de points cible tout en couvrant ses différentes zones. La seconde est que les commandes restent bornées et que les rapprochements dangereux soient peu fréquents. La troisième concerne la reproductibilité : une même configuration et une même graine doivent permettre de retrouver le même scénario, tandis que plusieurs graines doivent être utilisées pour éviter de conclure à partir d'un cas favorable.

Les critères liés à la robustesse doivent être évalués séparément. Tester plus de robots que lors de l'entraînement ne prouve pas à lui seul que le modèle est généralisable ; cela montre seulement qu'il peut être exécuté avec une taille différente. De même, retirer des robots teste une panne simplifiée, mais ne représente pas encore des capteurs erronés ou un robot immobilisé qui deviendrait un obstacle. Cette manière de préciser les critères de réussite permet de limiter les conclusions aux expériences effectivement réalisées.

1.7 Démarche générale du mémoire

La démarche suivie part d'un problème simple et augmente progressivement la difficulté. Il faut d'abord disposer d'un simulateur qui produit un voisinage cohérent et une dynamique stable. Sans cette étape, une mauvaise formation pourrait provenir du calcul des distances, de la mise à jour des positions ou de la visualisation, et non du modèle appris. Une règle locale de regroupement est donc utilisée comme contrôle initial. Elle n'est pas présentée comme la solution au problème de formation de formes ; elle sert à vérifier le socle de simulation.

La deuxième étape consiste à représenter la forme cible. Le choix d'un nuage de points permet de traiter une forme comme un ensemble de positions à couvrir plutôt que comme une image dont le réseau devrait prédire chaque pixel. Cette représentation est compatible avec un nombre limité de robots et permet d'utiliser une distance entre ensembles de points. Elle rend aussi explicite une contrainte pratique : si une forme comporte plus de

détails que le nombre de robots disponibles, certains détails ne pourront pas être rendus correctement.

La troisième étape est l'apprentissage de la politique GNN. L'objectif n'est pas de produire une trajectoire prédéfinie pour un robot particulier. Les robots commencent dans des positions aléatoires et appliquent tous les mêmes paramètres. L'apprentissage doit donc favoriser une loi qui réagit aux configurations locales rencontrées pendant les épisodes. Ce choix explique l'importance des graines, des scénarios variés et des évaluations séparées de l'entraînement.

Enfin, les résultats sont analysés à plusieurs niveaux. Une courbe de perte renseigne sur l'optimisation. Une image ou une animation renseigne sur la qualité géométrique observable. Des agrégats sur plusieurs épisodes renseignent sur la variabilité. Aucun de ces niveaux ne suffit isolément. Le mémoire adopte cette séparation pour éviter qu'une animation spectaculaire soit interprétée comme une validation statistique, ou qu'une moyenne favorable masque des échecs importants.

1.8 Organisation du mémoire

Le mémoire s'organise en cinq chapitres. Le chapitre 2 situe la formation décentralisée dans la robotique en essaim et présente le rôle des GNN dans le contrôle distribué. Le chapitre 3 décrit le simulateur, les formes cibles, le contrôleur, l'apprentissage, ainsi que l'architecture logicielle et les choix de reproductibilité. Le chapitre 4 présente les expériences et les résultats disponibles. Enfin, le chapitre 5 discute ces résultats, en synthétise les apports et ouvre sur les perspectives.

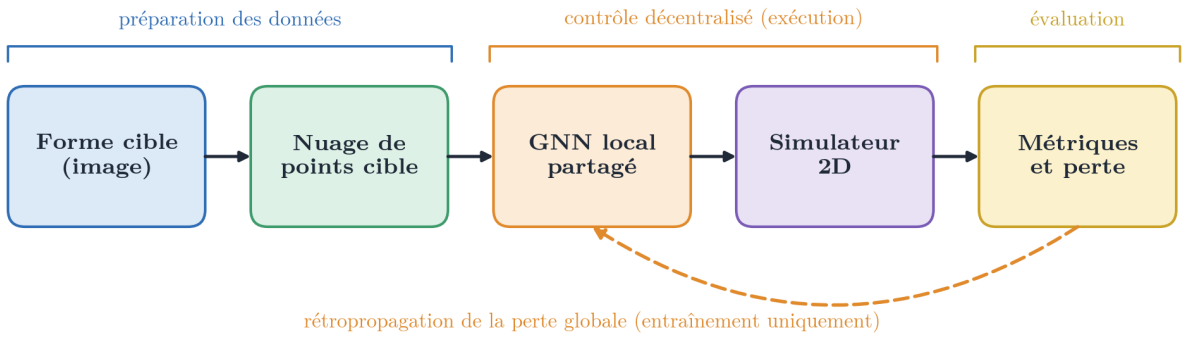


FIGURE 1.2 – Vue d'ensemble de la chaîne expérimentale. Une perte globale est utilisée pendant l'apprentissage ; lors de l'exécution, chaque robot applique une politique partagée à partir de son observation locale, de ses voisins et de la consigne de mission.

Chapitre 2

État de l'art

2.1 Robotique en essaim et contrôle décentralisé

La robotique en essaim s'intéresse à la coordination d'agents nombreux, homogènes ou faiblement hétérogènes, qui accomplissent collectivement une tâche sans dépendre d'un superviseur unique. La revue de Brambilla et al. souligne que cette approche associe généralement simplicité individuelle, redondance et interactions locales [3]. L'objectif n'est pas seulement de répartir une tâche entre plusieurs robots : il est de concevoir des règles locales dont l'effet collectif reste utile lorsque la taille du groupe, la topologie de communication ou la disponibilité des agents évoluent.

Les modèles comportementaux ont fourni des exemples fondateurs de cette idée. Le modèle de Reynolds repose sur trois mécanismes simples — séparation, alignement et cohésion — qui génèrent des déplacements collectifs crédibles sans plan global explicite [26]. Dans un cadre de contrôle multi-agent, Olfati-Saber formalise des lois de flocking qui combinent cohésion, évitement de collision et maintien de la connectivité à travers un graphe d'interactions [21]. Ces approches ont l'avantage de rendre les mécanismes de décision lisibles : un terme de répulsion produit un effet identifiable, de même qu'un terme d'attraction vers les voisins.

Leur limite est que le concepteur doit choisir la structure de la loi, les termes à combiner et les paramètres numériques. Une règle de regroupement peut amener les robots vers une zone commune, sans pour autant les répartir sur un contour précis. De la même manière, une règle bien réglée pour un cercle ne se généralise pas automatiquement à une lettre ou à un nombre. La formation de formes déplace donc le problème de la simple cohésion vers la construction distribuée d'une géométrie donnée.

2.2 Inspirations biologiques et exigences d'ingénierie

La robotique en essaim emprunte ses idées à l'observation des systèmes naturels : essaims d'abeilles, bancs de poissons, colonies de fourmis ou regroupements de bactéries. Camazine et al. montrent que de nombreux phénomènes collectifs — construction de nids, approvisionnement, régulation de la température — résultent de règles locales mises en œuvre par chaque individu, sans plan global ni individu dirigeant [4]. Trois enseignements en sont tirés et reviennent fréquemment dans la littérature robotique : la redondance, puisque la perte de quelques unités est partiellement absorbée ; l'absence de point unique de défaillance ; et un ajustement continu, le groupe réagissant à l'environnement par les mêmes règles à chaque instant.

Le modèle de Vicsek constitue le formalisme minimal de cette intuition [33]. Des particules auto-propulsées conservent une direction proche de celle de leurs voisins, avec un bruit aléatoire ; au-delà d'un seuil combinant densité et bruit, l'ensemble développe un ordre collectif. Ce résultat est intéressant pour deux raisons. D'abord, il établit qu'un comportement global peut naître d'une règle purement locale. Ensuite, il montre que cet ordre apparaît *par seuil* : il n'est pas garanti dans toutes les conditions, ce qui justifie de mesurer la structure du voisinage pendant l'exécution et non de la supposer.

Şahin propose de distinguer deux tâches de l'ingénierie de l'essaim : exploiter les sources d'inspiration biologique pour comprendre les mécanismes, puis les traduire en spécifications robotiques concrètes — tâche, coordonnée, mécanisme de communication [29]. Cette distinction est méthodologique et se retrouve dans le présent travail. L'analogie biologique suggère une structure locale et homogène ; elle ne dit pas comment une forme précise doit être représentée, ni comment sa qualité sera mesurée. Ces deux questions relèvent de la spécification du problème et sont traitées au chapitre 3.

Côté théorique, Cucker et Smale formalisent l'émergence d'un mouvement cohérent : sous des hypothèses de régularité du noyau d'interaction et de connectivité suffisante, les vitesses des agents convergent vers une direction commune [6]. Ces résultats supposent des interactions continues et des états comparables ; ils ne fournissent pas la géométrie finale recherchée ici. Ils établissent toutefois un principe qui gouverne la suite : l'émergence d'un comportement collectif dépend de conditions vérifiables sur le graphe d'interaction. Une règle locale, aussi judicieuse soit-elle, ne peut rien coordonner si le graphe se fragmente.

Deux conséquences pratiques découlent de cette revue. Premièrement, les interactions restent limitées au voisinage de communication, ce qui conserve l'hypothèse de localité. Deuxièmement, la structure du graphe — nombre de voisins, degrés extrêmes, connexité — doit être observée pendant l'évaluation, et non considérée comme un détail d'implémentation. Le passage de la cohésion à la formation de formes ajoute une exigence que l'alignement ne fournit pas : une géométrie de cible précise, qui fait l'objet de la section suivante.

2.3 Consensus, agrégation et convergence distribuée

Une famille voisine de travaux ne vise pas une géométrie mais un accord : amener les agents à partager une même valeur, une même décision ou un même état. Jadbabaie, Lin et Morse montrent que des règles fondées uniquement sur les voisins les plus proches suffisent à faire converger un groupe de mobiles autonomes vers un état commun, à condition que l'interaction soit suffisamment riche au cours du temps [14]. Olfati-Saber et Murray traitent le cas de topologies commutatives et de délais de communication [22], tandis que Ren et Beard établissent des conditions de convergence pour des graphes d'interaction variant pendant l'exécution [25]. Mesbahi et Egerstedt regroupent ces résultats dans un cadre graph-théorique commun, qui fournit le vocabulaire utilisé par la suite : graphe, degré, connectivité, laplacien [20].

Un pas de consensus discrète prend typiquement la forme d'une mise à jour moyennée : à chaque itération, un agent remplace sa valeur par une moyenne de sa propre valeur et de celles de ses voisins, pondérée par la présence des arêtes a_{ij} du graphe et le degré d_i de chaque nœud. La convergence est reliée à la connectivité du graphe au fil des itérations : si les communications permettent d'atteindre, directement ou par chaîne, l'ensemble des agents, l'accord peut se produire ; sinon, le groupe se scinde en composantes qui ignorent leurs états respectifs.

La figure 2.1 illustre ce mécanisme sur le cas le plus simple : un graphe linéaire de cinq agents. Chaque passe remplace la valeur d'un agent par la moyenne de son voisinage et la sienne propre. L'écart entre les valeurs extrêmes décroît à chaque passe, d'abord rapidement puis de plus en plus lentement : c'est la signature d'une convergence exponentielle, dont le taux dépend du spectre du laplacien du graphe. Deux conséquences directes se dégagent pour la suite du mémoire. D'une part, la convergence est un phénomène de groupe, et non la somme de convergences individuelles : aucun agent ne décide seul de la vitesse d'accord. D'autre part, la lenteur finale explique pourquoi les expériences du chapitre 4 mesurent un pas de convergence plutôt qu'un simple résultat final.

Ce cadre diffère du problème étudié dans ce mémoire à plusieurs égards. L'état partagé y est un scalaire ou un vecteur, la mise à jour est linéaire et la tâche est de s'accorder, non de produire une géométrie. Ici, la sortie est une commande de mouvement, la mise à jour est non linéaire et paramétrée, et la cible n'est pas négociée entre robots : elle est fournie par consigne. Le parallèle n'en reste pas moins méthodologiquement utile pour deux raisons. La première est diagnostique : lorsque l'essaim échoue, il faut pouvoir distinguer un échec de la loi de contrôle d'une rupture de connectivité, ce qui impose de surveiller le graphe de communication. La seconde est structurelle : l'agrégation des messages dans le contrôleur proposé est une version apprise d'une opération d'agrégation voisine de celle du consensus, avec la même propriété d'invariance par réordonnancement.

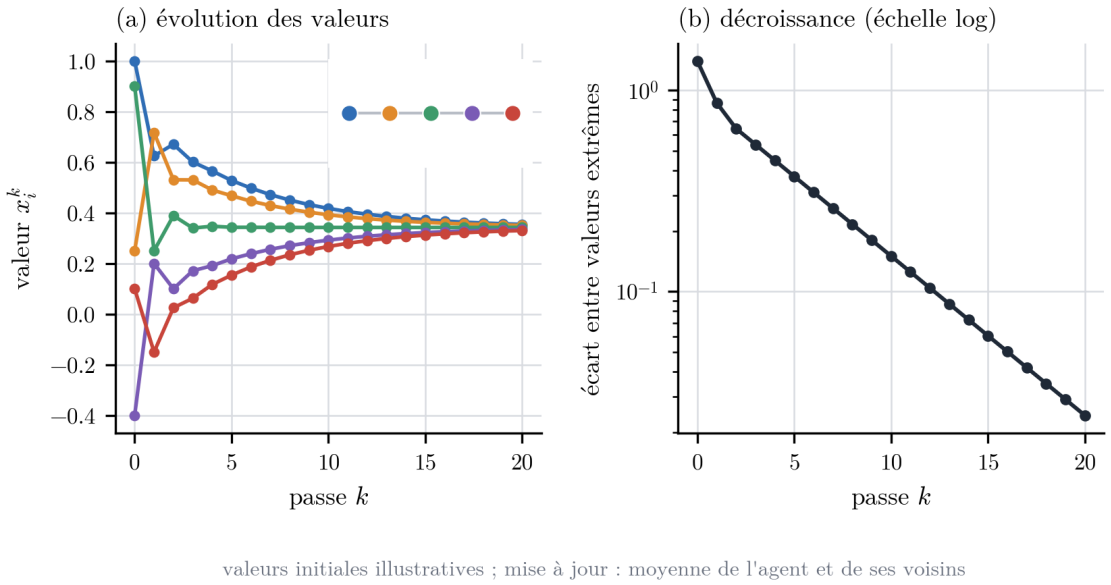


FIGURE 2.1 – Consensus sur un graphe linéaire de cinq agents. À gauche, évolution des valeurs à chaque passe ; à droite, décroissance de l'écart entre les valeurs extrêmes sur une échelle logarithmique.

2.4 Flocking et comportements locaux

Le flocking désigne un déplacement collectif dans lequel les agents restent groupés tout en conservant une direction de mouvement cohérente. Dans sa forme classique, il combine trois idées. La cohésion rapproche un agent du centre de ses voisins. La séparation produit une force opposée lorsque deux agents deviennent trop proches. L'alignement rapproche la vitesse d'un agent de la vitesse moyenne observée autour de lui. Ces mécanismes ne nécessitent pas de chef, car chaque robot applique la même règle à partir de son voisinage [26]. La figure 2.2 décompose visuellement ces trois contributions sur un essaim miniature ; le robot orange, dont on cherche le déplacement, pondère localement les trois flèches selon son propre voisinage.

Cette famille de règles est une référence utile pour deux raisons. D'abord, elle donne un comportement de base facilement interprétable : si les robots s'accumulent, on peut renforcer la séparation ; s'ils se dispersent, on peut renforcer la cohésion. Ensuite, elle montre que des interactions très simples suffisent à créer un comportement collectif. Dans ce projet, la règle de cohésion-répulsion joue ce rôle de contrôle : elle permet de vérifier que le simulateur et le calcul de voisinage produisent bien un regroupement avant d'introduire le GNN.

Le flocking ne résout toutefois pas directement la formation de forme. Un essaim peut rester cohérent et se déplacer de manière alignée tout en restant loin du contour demandé. Ajouter une attraction vers une cible est possible, mais il faut alors définir comment chaque robot choisit une zone du contour et comment éviter que tous convergent vers le même endroit. C'est précisément l'espace de décision que l'apprentissage cherche à explorer dans

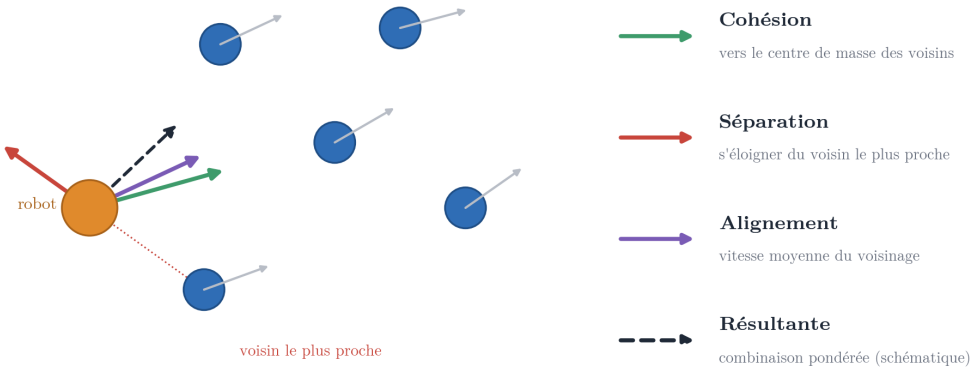


FIGURE 2.2 – Les trois contributions d'une règle de flocking appliquées au robot orange : attraction vers le centre de masse des voisins (cohésion), éloignement du voisin le plus proche (séparation) et alignement sur la direction moyenne du groupe.

ce mémoire. Le GNN ne remplace pas les idées de cohésion ou de séparation par magie ; il apprend une fonction de décision qui peut utiliser les vitesses, les distances et les messages du voisinage pour réduire la perte définie pendant l'entraînement.

2.5 Formation de formes : de la géométrie imposée à la consigne partagée

La formation de formes vise à faire converger les positions des robots vers une structure spatiale : une courbe, une formation rigide, une surface ou un nuage de points. Hsieh, Kumar et Chaimowicz étudient des contrôleurs décentralisés pour la génération de formes par essaims et mettent en évidence le rôle de l'information géométrique disponible localement [12]. Li et al. proposent une approche progressive de formation de formes, qui cherche à construire la cible à partir de décisions distribuées [18]. Ces travaux montrent que la forme elle-même doit être représentée d'une manière compatible avec les observations des agents.

Plusieurs représentations sont envisageables. Une cible peut être décrite par une équation analytique, par des positions de référence attribuées aux robots, par une carte, par des points de contour ou par des informations perceptives. Chaque choix modifie la difficulté du problème. Une affectation fixe robot–point fournit une information très forte, mais impose de résoudre l'association et peut limiter l'adaptation aux pannes. À l'inverse, une simple étiquette de forme contient peu de géométrie : elle est facile à partager, mais elle demande au contrôleur d'apprendre comment cette étiquette doit modifier le comportement.

Le présent projet utilise des nuages de points comme représentation de la cible. Cette

représentation évite de lier durablement un robot particulier à un point particulier ; la qualité de formation est mesurée par la proximité entre les deux ensembles de points. Elle doit cependant être interprétée correctement. Le nuage cible est une consigne de mission définie avant le déploiement ; son accès n'est pas une communication entre robots. Lorsque la configuration utilise le vecteur vers le point de contour le plus proche, elle suppose une forme de carte ou de requête géométrique locale. Cette hypothèse facilite la formation de contours complexes, mais elle ne doit pas être confondue avec une perception autonome de la forme dans un environnement réel.

2.6 Réseaux de neurones sur graphes et passage de messages

Un essaim à voisinage local se modélise naturellement par un graphe dynamique $\mathcal{G}^t = (\mathcal{V}, \mathcal{E}^t)$. Chaque robot est un nœud ; une arête relie deux robots lorsqu'ils peuvent échanger des informations au pas de temps t. Les réseaux de neurones sur graphes étendent les réseaux neuronaux à ce type de données relationnelles. Dans le cadre du passage de messages, un nœud produit ou reçoit des messages calculés à partir des états des nœuds reliés et des attributs d'arête, puis agrège ces messages avant de mettre à jour sa représentation [9]. Le cadre plus général des *graph networks* formalise de la même façon les mises à jour de nœuds, d'arêtes et, éventuellement, d'attributs globaux [2].

Deux propriétés rendent ce formalisme particulièrement adapté au contrôle d'essaim. D'abord, les paramètres sont partagés : la même fonction de message et la même fonction de décision peuvent être utilisées pour chaque robot. Ensuite, une agrégation par somme, moyenne ou maximum est invariante à l'ordre des voisins. Un robot peut donc recevoir un nombre différent de messages au cours du temps sans qu'il soit nécessaire de modifier la taille de l'entrée du réseau. Ces propriétés ne garantissent pas à elles seules une bonne généralisation, mais elles introduisent un biais inductif cohérent avec le caractère relationnel et variable d'un essaim.

Dans le contrôleur implémenté, l'agrégation est une moyenne. Chaque voisin j envoie d'abord un message m_{ij} calculé à partir de son état, de l'état du robot i et de leur géométrie relative. Le robot i calcule ensuite la moyenne de ces messages sur son voisinage $\mathcal{N}(i)$. La moyenne a une conséquence pratique : la magnitude de l'information agrégée ne croît pas mécaniquement avec le nombre de voisins, contrairement à une somme non normalisée. Un robot au centre d'un groupe dense et un robot proche du bord peuvent ainsi être traités avec des échelles numériques plus comparables. Lorsqu'un robot n'a aucun voisin, l'agrégat est remplacé par un vecteur nul, puis le réseau produit tout de même une commande valide à partir de son état propre et de la consigne.

Le passage de messages possède également une limite structurelle : avec L tours

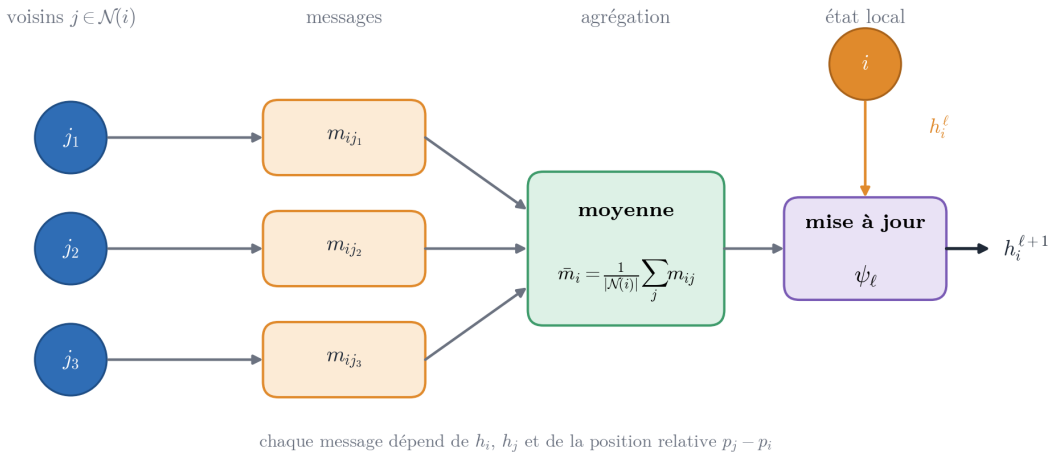


FIGURE 2.3 – Principe d'agrégation employé dans le contrôleur. Les messages individuels sont calculés pour chaque voisin, puis leur moyenne est utilisée pour mettre à jour l'état local du robot.

d'échange, l'état d'un robot ne peut intégrer directement que les informations disponibles dans son voisinage à au plus L sauts, pour la topologie considérée. La figure 2.4 rend cette portée concrète : le robot i au centre reçoit, après un tour, les informations de son anneau immédiat ; après deux tours, celles de l'anneau suivant ; au-delà, l'information est structurellement absente de son état. Augmenter L peut étendre la portée informationnelle, mais augmente aussi le coût de calcul et peut rendre l'apprentissage plus difficile. Un GNN ne transforme donc pas un système local en contrôleur centralisé : il fournit une manière paramétrable d'exploiter des communications restreintes.

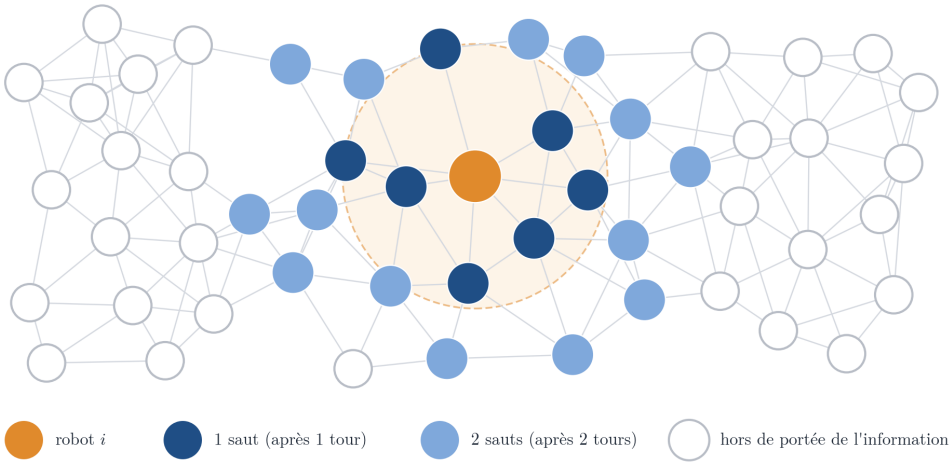


FIGURE 2.4 – Portée de l'information selon le nombre de tours de messages. Chaque anneau correspond à un saut supplémentaire dans le graphe de communication ; les expériences du mémoire utilisent deux tours.

2.7 Variants des GNN et justification de l'agrégation

Scarselli et al. proposent le modèle initial de réseau de neurones sur graphe, défini par la propagation d'états jusqu'à un point fixe puis par un décodeur [30]. Les architectures contemporaines privilégient une profondeur finie, entraînable par rétropropagation classique. Wu et al. organisent la littérature en grandes familles — modèles récurrents, convolutifs, auto-encodeurs et à attention — et soulignent que le choix du mécanisme d'agrégation y tient une place centrale [34].

L'essaim impose une contrainte forte sur cette fonction d'agrégation : le nombre et l'ordre des voisins varient d'un robot à l'autre et d'un pas à l'autre. L'agrégation doit donc être une fonction symétrique de l'ensemble des messages reçus. Zaheer et al. établissent que cette invariance — exprimée par une somme sur les ensembles — est une condition de représentation des données non ordonnées [35]. La moyenne employée dans ce mémoire satisfait cette condition et, contrairement à une somme non normalisée, maintient la même échelle numérique quel que soit le nombre de voisins.

Deux familles de variants sont souvent utilisées comme alternatives. Les réseaux convolutifs sur graphes normalisent les contributions par le degré des nœuds [16] ; les réseaux à attention apprennent au contraire des poids distincts pour chaque voisin [32]. L'attention présente un intérêt évident pour ce sujet : elle permettrait d'accorder moins de poids à un voisin aberrant, par exemple un robot défaillant qui transmettrait une information incohérente. Elle introduit toutefois des paramètres supplémentaires, une sortie d'attention à interpréter et un coût de calcul plus élevé. La moyenne, plus sobre, laisse au réseau le soin d'apprendre les pondérations utiles à partir des états eux-mêmes.

Ce choix s'inscrit dans la logique du prototype : il convient d'isoler l'apport du passage de messages avant d'ajouter des mécanismes plus riches. Il comporte une limite assumée. Lorsqu'un voisin transmet une information faussée, sa contribution pèse exactement $1/|\mathcal{N}(i)|$ et aucun filtre ne la corrige. Une étude de robustesse au bruit devra comparer la moyenne à une agrégation robuste — par exemple tronquée ou pondérée par l'attention — avant de conclure sur le comportement du contrôleur en présence de capteurs défaillants.

2.8 Induction, partage des paramètres et changement d'échelle

La possibilité d'appliquer un même modèle à une taille d'essaim différente est souvent associée au caractère inductif des architectures de graphe. Dans GraphSAGE, Hamilton, Ying et Leskovec apprennent une fonction qui agrège les attributs d'un voisinage local au lieu d'apprendre un vecteur propre à chaque nœud [11]. Le problème traité dans cet article est différent de la robotique, mais le principe est utile ici : une politique ne doit pas dépendre de l'identifiant d'un robot ni d'une liste d'entrée de longueur fixe.

Cette propriété ne doit pas être présentée comme une garantie de performance. Lorsque le nombre de robots augmente, la densité locale, la connectivité et le nombre moyen de voisins peuvent aussi changer. Une politique peut donc être techniquement exécutable avec cent robots tout en produisant une formation moins régulière que pour vingt robots. Le passage à l'échelle doit être mesuré, avec une cible et des conditions initiales adaptées, plutôt que supposé à partir de l'architecture.

Les travaux sur les contrôleurs GNN décentralisés insistent également sur cet écart entre propriété architecturale et validation expérimentale. Gama et al. proposent d'apprendre des contrôleurs sur des graphes en exploitant le partage des paramètres et l'invariance aux permutations, puis évaluent le transfert vers des équipes plus grandes et des scénarios nouveaux [8]. Dans le cas de la formation triangulaire étudiée par Jiang et al., une politique entraînée avec cinq robots est testée avec des équipes de quatre à neuf robots [15]. Ces résultats motivent les essais de passage à l'échelle du présent projet, mais ne dispensent pas d'un protocole spécifique aux formes et aux hypothèses retenues ici.

2.9 Apprentissage de contrôleurs décentralisés par GNN

Les GNN ont été étudiés comme politiques de contrôle lorsque les agents communiquent sur un graphe. Tolstaya et al. montrent qu'un contrôleur GNN peut être entraîné pour des tâches de robotique en essaim tout en conservant une exécution locale sur un graphe de communication mobile [31]. Gama, Tolstaya et Ribeiro analysent plus largement les GNN comme contrôleurs décentralisés et mettent l'accent sur les propriétés de partage des paramètres et de transfert entre topologies [7]. Ces travaux motivent l'usage d'une architecture relationnelle plutôt qu'un réseau dense dont l'entrée supposerait une liste fixe de robots.

Pour la formation de groupe, Jiang, Huang et Guo proposent une méthode de formation décentralisée de bout en bout fondée sur un GNN [15]. Ce type de travail rapproche directement l'apprentissage du problème étudié ici : un contrôleur doit convertir des observations locales et des informations de voisinage en actions coordonnées. Les approches diffèrent toutefois selon les capteurs disponibles, la forme à produire, le mode d'apprentissage et la manière dont la cible est communiquée. Une comparaison rigoureuse ne peut donc pas se limiter à constater que plusieurs méthodes utilisent un GNN ; elle doit examiner quelles informations sont réellement accessibles à chaque robot au moment de l'exécution.

Deux stratégies d'apprentissage sont courantes. La première consiste à apprendre à imiter un contrôleur expert ou centralisé ; elle peut produire un signal d'entraînement informatif, mais la politique finale doit être évaluée sans accéder à l'information centrale

de l'expert. La seconde consiste à optimiser directement le comportement dans un simulateur différentiable ou par renforcement. Le projet retient ici l'apprentissage direct à travers un simulateur différentiable, afin d'optimiser une mesure de proximité à la forme et des contraintes de sécurité. Cette décision favorise la reproductibilité et la simplicité du prototype ; elle ne constitue pas une comparaison expérimentale entre les paradigmes d'apprentissage.

2.10 Apprentissage de contrôle : imitation, simulation différentiable, multi-agents

La place du concepteur dans la loi de contrôle est aussi un enjeu d'apprentissage. Une première génération de travaux montre qu'il est possible d'apprendre directement une fonction d'observation vers commande plutôt que d'écrire cette fonction. ALVINN, de Pomerleau, conduit un véhicule à partir d'observations issues de données humaines [23] et pose le principe d'une politique apprise de bout en bout. Ross, Gordon et Bagnell précisent ensuite une limite importante de l'imitation : si la politique apprise génère des états différents de ceux de l'expert, l'expert n'est plus consulté dans les situations qui comptent, et l'écart s'accumule [27]. L'entraînement doit donc se dérouler dans les états réellement visités par la politique.

Ce constat éclaire le choix de ce mémoire. La politique n'imite pas de trajectoire : elle est entraînée dans l'épisode qu'elle-même produit, ce qui évite le décalage de distribution décrit par ces travaux. Le signal d'apprentissage vient d'une mesure de qualité de forme calculée en fin d'épisode et remontée à travers la simulation. Cette idée s'inscrit dans la lignée des travaux de physique différentielle appliqués au contrôle [1] et des environnements de programmation différentielle [13]. Elle repose ici sur une dynamique suffisamment simple pour être dérivable, ce qui permet d'obtenir un gradient de faible variance sans estimer les dérivées par échantillonnage, comme le ferait une méthode de gradient stochastique.

Les travaux d'apprentissage multi-agents proposent d'autres architectures, motivées par la difficulté de créditer une action individuelle dans un épisode collectif. Lowe et al. combinent un acteur décentralisé par agent avec un critique ayant accès à l'information partagée [19] ; Rashid et al. factorisent une fonction de valeur commune à partir de contributions individuelles contraintes [24]. Ces méthodes supposent des agents potentiellement hétérogènes et des récompenses individuelles. Dans le présent projet, le partage des paramètres élimine en grande partie cette difficulté : un seul jeu de poids décrit tous les robots, le problème se ramène à l'optimisation d'une unique politique. Cette simplification a un coût : le contrôleur ne peut pas attribuer de rôle différencié aux robots du bord et à ceux du centre, et toute différence de comportement doit émerger des observations locales.

Le mode d'apprentissage retenu — gradient direct à travers un simulateur différentiable — est donc motivé par le coût, la variance du gradient et la reproductibilité. Il ne constitue pas une preuve de supériorité sur l'imitation ou l'apprentissage par renforcement : ces paradigmes ne sont pas comparés expérimentalement dans ce mémoire, et la comparaison reste ouverte.

2.11 Lecture critique des comparaisons

La comparaison avec une méthode classique exige que les méthodes reçoivent des informations comparables. Une règle de cohésion et de répulsion qui ne connaît que les voisins peut être une bonne baseline pour le regroupement, mais elle ne dispose pas de la même information qu'un GNN recevant une consigne de forme. Si le GNN obtient une meilleure distance à une cible circulaire dans cette situation, le résultat établit surtout que la consigne et la politique entraînée sont adaptées à la tâche. Il ne permet pas encore d'isoler l'effet du GNN lui-même.

Une étude plus complète devrait comporter au moins trois contrôles. Le premier est une règle manuelle qui reçoit une description de cible comparable. Le deuxième est un réseau sans communication entre voisins, mais avec la même consigne de forme et une taille de modèle similaire. Le troisième est une variante de profondeur différente, par exemple un ou trois tours de messages. Ces ablations ne sont pas accessoires : elles permettent de répondre à la question précise de l'apport de la communication et de l'architecture relationnelle.

2.12 De la règle écrite à la règle apprise

Une différence importante entre les méthodes classiques et le GNN concerne la place du concepteur. Dans une loi de potentiel, le concepteur écrit explicitement les forces d'attraction, de répulsion et éventuellement d'alignement. Il choisit aussi leurs gains et les distances à partir desquelles elles agissent. Cette démarche apporte de la lisibilité : lorsque le système se comporte mal, il est possible d'identifier un terme et de le modifier. En contrepartie, l'ajout d'une forme complexe ou d'une nouvelle contrainte peut demander une nouvelle dérivation de la loi et plusieurs réglages manuels.

Dans une approche apprise, le concepteur ne choisit pas directement la réaction exacte à chaque configuration. Il choisit les informations accessibles, la structure du réseau, les objectifs à optimiser et les scénarios d'entraînement. Les paramètres de la politique sont ensuite ajustés à partir des épisodes simulés. Le déplacement de responsabilité est réel : une mauvaise observation locale, une perte mal définie ou un entraînement trop court peuvent empêcher le modèle d'apprendre, même si le GNN est bien implémenté.

Il serait toutefois simpliste d'opposer une méthode explicable à une méthode opaque. Les deux familles dépendent d'hypothèses de modélisation. Une règle de flocking suppose qu'attraction, répulsion et alignement sont suffisants pour la mission choisie. Le GNN suppose que les exemples rencontrés pendant l'entraînement couvrent les situations utiles et que la perte favorise réellement la formation souhaitée. Le mémoire ne cherche donc pas à déclarer une famille universellement meilleure. Il étudie dans quelles conditions une règle apprise, exécutée localement, peut être une alternative raisonnable à une règle manuelle.

2.13 Consigne, perception et déploiement

Les publications sur le contrôle décentralisé ne font pas toutes les mêmes hypothèses sur les capteurs. Certaines utilisent les positions absolues des robots, d'autres des mesures de distance et d'angle, d'autres encore des cartes d'occupation issues de capteurs embarqués. Jiang et al. utilisent par exemple des observations LiDAR transformées en cartes locales, alors que le prototype de ce mémoire travaille avec des états simulés plus simples [15]. Cette différence doit être prise au sérieux : une architecture qui fonctionne avec une position exacte ne garantit pas qu'elle fonctionnera après remplacement par un capteur bruité.

Le choix du présent projet est intermédiaire. La communication entre robots est locale, mais la forme cible est fournie comme une consigne partagée. Cette hypothèse est justifiée pour une mission préparée à l'avance, par exemple une formation lumineuse ou une distribution planifiée. Elle serait moins adaptée à une tâche où la forme doit être découverte dans l'environnement. Dans ce second cas, il faudrait intégrer une perception locale, une carte distribuée ou un mécanisme de consensus sur la cible.

Cette distinction permet aussi de préciser le sens du mot décentralisé. Un contrôleur peut être décentralisé au moment de l'exécution tout en ayant été entraîné de manière centralisée. Il peut aussi utiliser une consigne commune sans recevoir les états distants de tous les robots. Ce qui serait contraire à l'hypothèse de travail serait de calculer, à chaque pas, une affectation globale des robots à la cible et de la transmettre à chacun. Le prototype évite cette opération pendant l'exécution, mais la consigne géométrique reste une hypothèse forte qui devra être discutée dans toute extension vers le réel.

2.14 Synthèse comparative et positionnement

Le tableau 2.1 réunit les familles d'approches discutées jusqu'ici. Il est volontairement organisé selon l'information disponible au moment de l'exécution, car cette information conditionne directement la nature de la décentralisation revendiquée. Les capacités de passage à l'échelle y sont notées *structurelles* lorsqu'elles découlent de l'architecture (partage des paramètres, agrégation invariante) et *mesurées* lorsqu'elles ont fait l'objet d'une

évaluation expérimentale.

TABLE 2.1 – Comparaison des grandes familles d'approches pour la formation de formes décentralisée.

Famille	Information à l'exécution	Apprentissage	Passage à l'échelle	Limite principale
Règles locales (flocking, cohésion–répulsion)	Voisins à portée	Aucun	Structurel	Pas de géométrie précise
Contrôle centralisé	État complet de l'essaim	Planification, affectation	Limité par la communication	Point unique de défaillance
Affectation robot–point de cible	Cible et affectation	Variable, souvent nul	Sensible au nombre d'agents	Association fragile en cas de panne
GNN décentralisé + imitation	Voisins, démonstrateur	Imitation	Inductif, à vérifier	Dépend de la qualité de l'expert [8]
GNN décentralisé + renforcement	Voisins, récompense	Renforcement multi-agents	Variable	Coût d'échantillonnage, non-stationnarité [19, 24]
Prototype de ce mémoire	Voisins et consigne géométrique locale	Gradient direct sur simulateur	Mesuré dans le chapitre 4	Consigne forte, simulation simplifiée

Le positionnement du mémoire se lit à partir de cette dernière ligne. L'apport n'est pas une nouvelle théorie de la formation distribuée, ni une architecture de GNN inédite. Il consiste en une chaîne expérimentale complète et reproductible qui relie un simulateur de voisinage, un jeu de formes symboliques, une politique partagée, un apprentissage différentiable et un protocole d'évaluation comportant pannes et changements d'échelle. Les travaux antérieurs couvrent séparément plusieurs de ces briques : la formation de formes par contrôle décentralisé [12, 18], l'apprentissage de contrôleurs GNN [31, 7], ou encore la formation de groupe de bout en bout [15]. Le point d'appui du présent travail est de les assembler dans une même chaîne et de rendre chaque hypothèse — consigne, portée, taille d'essaim, seuil de succès — explicite et rejouable.

Cette lecture impose aussi une exigence de comparaison. Les baselines doivent recevoir une information de consigne comparable, sinon l'écart mesuré sera imputable à l'information et non au contrôleur. Le chapitre 4 applique cette règle en séparant les résultats pilotes, qui comparent encore deux régimes d'information différents, des comparaisons qui restent à conduire.

2.15 Réseaux cellulaires et positionnement du mémoire

L'intitulé initial du mémoire fait référence aux réseaux de neurones cellulaires, formalisme introduit par Chua et Yang, dont chaque cellule n'interagit qu'avec son voisinage immédiat [5]. L'idée commune est locale : des unités identiques évoluent selon leurs interactions avec un voisinage limité. Dans l'essaim étudié, le voisinage n'est pas une grille fixe ; il dépend des positions et de la portée de communication. De plus, les messages sont produits par un réseau appris. L'implémentation est donc plus précisément décrite comme un GNN à passage de messages. La référence aux réseaux cellulaires doit être comprise comme une inspiration de contrôle local homogène, non comme l'implémentation littérale d'un réseau cellulaire classique.

Le positionnement de ce mémoire est expérimental. Il ne revendique pas une nouvelle théorie de la formation distribuée ni une supériorité générale du GNN. Il met en place une chaîne reproductible qui relie un simulateur 2D, des formes cibles, une politique partagée, un apprentissage différentiable et des mesures de robustesse. La contribution recherchée est double : vérifier qu'une telle politique peut former des cibles dans un cadre contrôlé, puis identifier les conditions qui limitent cette capacité.

Cette prudence implique deux exigences pour les expériences. Premièrement, les baselines doivent recevoir une information de consigne comparable à celle du GNN. Une règle de simple regroupement constitue un contrôle utile du simulateur, mais ne suffit pas à démontrer un gain dû à l'apprentissage de la forme. Deuxièmement, les résultats visuels doivent être complétés par des métriques, plusieurs graines et une analyse des échecs. Les chapitres suivants décrivent le protocole conçu dans cette perspective.

Chapitre 3

Méthode et architecture logicielle

3.1 Formulation du problème

Le problème traité peut être posé sans référence à une architecture particulière. Soit un ensemble de N robots actifs évoluant dans une arène bornée. Chaque robot observe son propre état, l'état de ses voisins à portée et une consigne de mission partagée ; il produit une commande de mouvement, sans accéder à l'état complet du groupe. Le tableau 3.1 réunit les symboles employés dans l'ensemble du mémoire.

Table 3.1 – Principaux symboles utilisés pour décrire l'essaim, la politique et l'évaluation.

Symbole	Signification
N	nombre de robots actifs de l'épisode
T	nombre de pas de simulation d'un épisode
Δt	pas de temps numérique
p_i^t, v_i^t	position et vitesse du robot i au pas t
u_i^t	commande de vitesse produite par le contrôleur
r_c	rayon de communication
$\mathcal{N}(i)$	voisins du robot i dans le graphe courant
$\mathcal{G}^t$	graphe de communication au pas t
h_i^ℓ, m_{ij}^ℓ	état latent et message au tour ℓ
g_i	vecteur vers le point de contour cible le plus proche
P, Q	positions finales des robots et nuage de points cible
θ	paramètres partagés de la politique
$\lambda_f, \lambda_c, \lambda_u, \lambda_r$	poids des quatre termes de la perte
τ	seuil de distance associé au taux de succès

Un épisode est défini par un triplet (cible Q, positions initiales p^0, graine aléatoire). La politique π_θ est appliquée de façon indépendante et simultanée à chaque robot actif pendant T pas. L'objectif d'apprentissage s'écrit alors :

$$\theta^\star = \arg\min_\theta \; \mathbb{E}_{(Q,\,p^0)\sim\mathcal{D}}\Big[\mathcal{L}\big(\pi_\theta;\, Q,\, p^0\big)\Big], \tag{3.1}$$

où $\mathcal{D}$ désigne la distribution des cibles et des départs utilisés pendant l'entraînement. La perte agrège quatre termes aux rôles distincts :

$$\mathcal{L} = \lambda_f \, d_C(P, Q) + \lambda_c \, \mathcal{L}_{col} + \lambda_u \, \mathcal{L}_{move} + \lambda_r \, \mathcal{L}_{cov}. \tag{3.2}$$

Le premier terme mesure l'adéquation à la cible, le second pénalise les rapprochements dangereux, le troisième décourage les commandes saturées et le quatrième rapproche la répartition de celle de la cible. Les poids numériques sont fixés dans les fichiers de configuration et sont identiques pour tous les robots ; ils sont rappelés dans le tableau 3.2.

Deux contraintes accompagnent la minimisation. La première est bornée : la norme de chaque commande reste inférieure ou égale à $v_{\max}$. La seconde est structurelle : aucun terme de l'équation (3.1) ne requiert l'état d'un robot hors voisinage. Cette contrainte est celle de l'exécution, mais elle guide aussi l'entraînement, car la perte est évaluée sur des épisodes où chaque robot ne reçoit effectivement que les messages du graphe courant.

3.2 Simulateur et hypothèses

On considère N robots évoluant dans une arène carrée bidimensionnelle. L'état du robot i au pas t comprend sa position $p_i^t \in \mathbb{R}^2$ et sa vitesse $v_i^t \in \mathbb{R}^2$. Le contrôleur produit une commande bornée u_i^t, puis le simulateur applique une dynamique à vitesse directe : la position est mise à jour par $p_i^{t+1} = p_i^t + \Delta t \, u_i^t$, la commande restant bornée par $\|u_i^t\| \leq v_{\max}$. La configuration utilisée pour les premiers entraînements comporte 20 robots dans une arène de 10×10, un pas de temps de 0,3 et une vitesse maximale de 1. Ces valeurs servent à stabiliser l'expérimentation ; elles ne sont pas interprétées comme les paramètres d'une plateforme physique particulière.

Le graphe de communication $\mathcal{G}^t = (\mathcal{V}, \mathcal{E}^t)$ est recalculé à chaque pas. Une arête est présente entre i et j lorsque $\|p_i^t - p_j^t\| \leq r_c$, avec $r_c = 2{,}5$ dans la configuration de référence. Ainsi, la topologie évolue avec les déplacements. Un robot en panne cesse de se déplacer et n'émet plus de messages ; il est retiré des calculs de la politique des robots actifs. Cette première version adopte un repère commun et des mesures parfaites. L'absence de repère propre, de délai ou de bruit constitue une limite discutée au chapitre 4.

3.3 Formes cibles

Le jeu de données contient sept formes géométriques, les 26 lettres de l'alphabet, les dix chiffres et un ensemble de nombres à trois caractères. Chaque image est convertie en un nuage de points puis normalisée dans un cadre commun. Pour une exécution donnée, un sous-ensemble de points de la cible est sélectionné afin de constituer la référence de formation. Cette représentation évite d'imposer une correspondance fixe entre l'indice

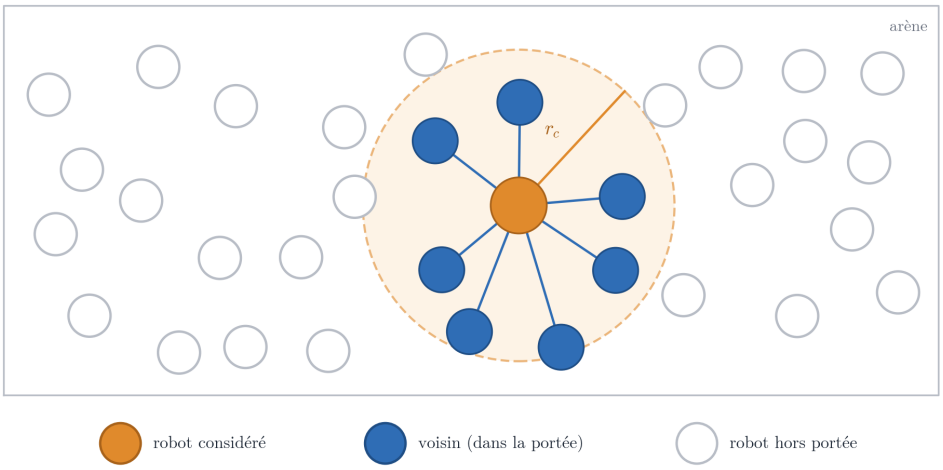


FIGURE 3.1 – Schéma d'un essaim. Le robot orange ne peut échanger qu'avec les robots situés dans sa portée de communication ; les robots distants ne sont pas accessibles directement.

d'un robot et l'indice d'un point cible.

La consigne disponible à tous les robots comporte un identifiant de forme et le nuage de points associé. À partir de sa propre position, un robot interroge ce nuage et obtient le vecteur vers le point de contour le plus proche. Cette information locale de géométrie est plus riche qu'un identifiant seul : elle indique dans quelle direction se trouve le trait à couvrir, sans fournir la position des autres robots. L'identifiant et le nuage de points ne sont pas des messages provenant d'un superviseur au cours de l'exécution : ils représentent une mission définie avant le déploiement. Le choix d'une consigne géométrique partagée facilite l'étude de la formation, mais il introduit une hypothèse qui devra être remplacée à terme par une perception, une carte ou une balise localement observable.

3.4 Génération des nuages de points cibles

Les formes sont produites par une chaîne déterministe, séparée de l'entraînement. Chaque cible est d'abord rendue comme une image bitmap de 128×128 pixels à partir d'une police de caractères ou d'une routine de tracé géométrique. Le seuillage utilise la valeur médiane de la plage de gris, ce qui évite d'avoir à régler un seuil par forme. Les pixels retenus constituent une première discrétisation du trait, redondante et irrégulière puisque la grille de l'image impose son propre maillage.

Un sous-ensemble régulier est ensuite extrait par échantillonnage par point le plus éloigné *(farthest-point sampling)*. La première graine de l'échantillonnage est tirée à partir de la graine de configuration ; les points suivants sont choisis, à chaque itération, comme le point du masque le plus éloigné des points déjà retenus. Contrairement à un tirage uniforme, cette procédure évite les grappes et les zones vides : elle répartit les points sur

l'ensemble du contour. Deux résolutions sont employées : 20 points pour le jeu standard et 50 points pour la campagne dédiée aux formes complexes.

Les points sont enfin ramenés dans un cadre commun centré sur l'arène et compressés à l'échelle du rayon de cible configuré. Cette normalisation garantit qu'une lettre étroite et un cercle large présentent la même amplitude pour le contrôleur, et que la perte de forme reste comparable d'une cible à l'autre. Chaque forme est sauvegardée sous forme de tableau numérique, accompagnée de son image d'origine et d'une ligne d'index ; une planche de contrôle réunit toutes les formes du jeu pour permettre une inspection visuelle avant tout entraînement.

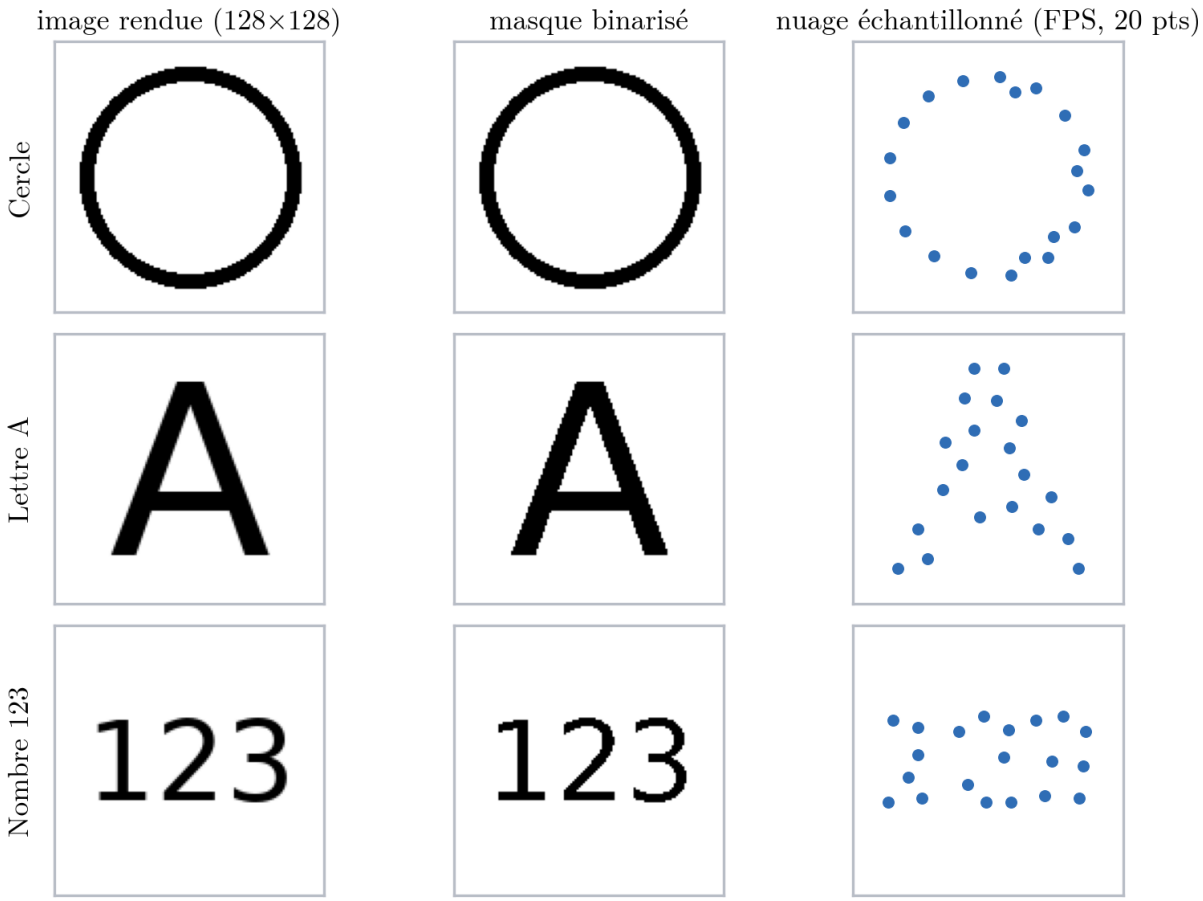


FIGURE 3.2 – Chaîne de préparation d'une cible : image rendue, masque binarisé, puis nuage de points échantillonné et normalisé. L'exemple provient du jeu de formes standard.

Cette chaîne comporte deux conséquences qu'il faut garder à l'esprit lors de l'analyse. D'une part, la résolution du trait et le nombre de robots sont liés : si une forme présente davantage de détails que l'essaim ne dispose de points, certains détails ne peuvent être représentés. D'autre part, l'échantillonnage étant fixé avant l'épisode, la même cible fournit la même configuration de référence pour tous les robots ; la consigne est donc bien identique pour tout l'essaim.

3.5 Contrôleur GNN

Chaque robot encode sa vitesse, le vecteur g_i vers le contour cible le plus proche et l'identifiant de forme. Son état latent initial est noté h_i^0. À la couche ℓ, le message de j vers i dépend des deux états et de la position relative :

$$m_{ij}^{\ell} = \phi_{\ell}\left(h_i^{\ell}, h_j^{\ell}, p_j - p_i\right), \qquad \bar{m}_i^{\ell} = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} m_{ij}^{\ell}. \tag{3.3}$$

Lorsque le voisinage est vide, l'agrégat est remplacé par le vecteur nul. Une fonction de mise à jour ψ_{ℓ} produit ensuite $h_i^{\ell+1} = \psi_{\ell}(h_i^{\ell}, \bar{m}_i^{\ell})$. Deux tours de passage de messages et une dimension cachée de 32 sont employés dans les expériences initiales. Un décodeur partagé transforme l'état final en commande de vitesse, puis une non-linéarité borne cette commande.

Le partage des paramètres garantit que la politique ne dépend pas de l'identité d'un robot. L'agrégation par moyenne la rend également insensible à l'ordre des voisins. En revanche, la portée effective de l'information reste limitée : avec deux tours, un robot ne résume directement que son voisinage à deux sauts pour la topologie considérée à cet instant.

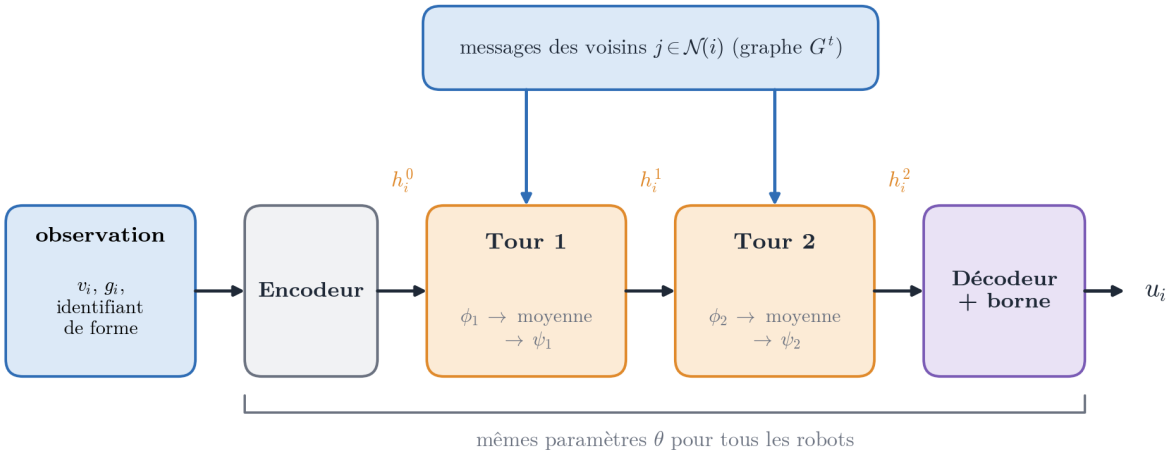


FIGURE 3.3 – Calcul local réalisé pour un robot lors d'un tour de passage de messages. Les mêmes blocs de calcul et les mêmes paramètres sont partagés dans l'essaim.

3.6 Apprentissage

L'entraînement est direct à travers le simulateur différentiable. Si P désigne l'ensemble des positions finales des robots actifs et Q le nuage de points cible, la qualité de forme est

mesurée par une distance de Chamfer symétrique :

$$d_C(P,Q) = \frac{1}{|P|}\sum_{p\in P}\min_{q\in Q}\|p-q\|^2 + \frac{1}{|Q|}\sum_{q\in Q}\min_{p\in P}\|q-p\|^2. \tag{3.4}$$

La fonction optimisée ajoute à ce terme une pénalité lorsque deux robots franchissent une distance de sécurité, une faible pénalité sur l'amplitude des commandes et une régularisation de couverture. Cette dernière compare les statistiques de distances au plus proche voisin de l'essaim à celles de la cible : elle réduit le risque qu'une distance de Chamfer faible soit obtenue par un amas de robots sur une fraction seulement de la forme. Les poids utilisés sont explicitement versionnés dans chaque configuration.

La figure 3.4 donne une lecture géométrique de l'équation (3.4). Chaque robot (orange) est relié au point cible (bleu) le plus proche, et inversement. La pénalité de couverture intervient précisément sur le cas que ce schéma masque : si tous les robots convergeaient vers un seul arc du contour, chaque robot trouverait bien un point cible proche — le premier terme de l'équation resterait faible — mais la moitié des points cibles ne serait reliée à aucun robot, et le second terme exploserait.

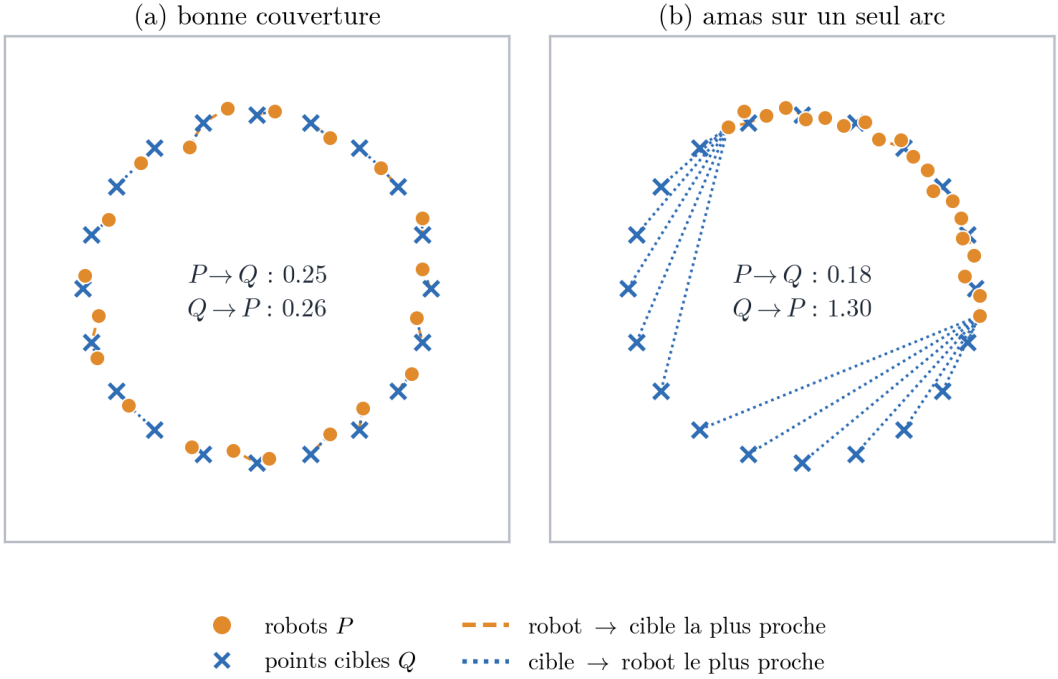


FIGURE 3.4 – Lecture géométrique de la distance de Chamfer symétrique : en pointillés, les appariements vers le voisin le plus proche, pris dans les deux sens entre les robots P et les points cibles Q.

La figure 3.5 réunit les quatre termes optimisés. Ils répondent à des échecs distincts : sans terme de forme, l'essaim n'a aucune raison de se déplacer ; sans pénalité de collisions, il pourrait se comprimer sur le contour ; sans pénalité de mouvement, les commandes pourraient osciller ; sans régularisation de couverture, une partie du contour resterait

vide.

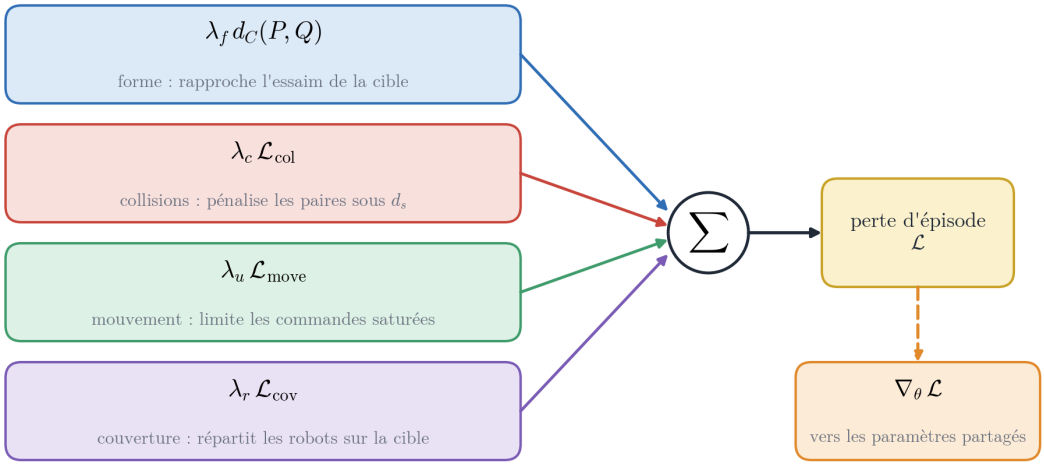


FIGURE 3.5 – Composition de la fonction de perte. Les quatre termes, pondérés par les coefficients λ versionnés dans les configurations, sont sommés en une seule perte d'épisode rétropropagée vers les paramètres partagés.

Un curriculum réduit introduit successivement les géométries, les chiffres, quelques lettres, puis deux nombres. La figure 3.6 montre un exemple de cible pour chacune des quatre étapes : le nombre de cibles vues par le contrôleur croît de 3 à 11, et chaque étape conserve les cibles des étapes précédentes dans son lot. Une seconde configuration, dédiée aux formes complexes, porte l'essaim et le nuage cible à 50 points, augmente la durée des épisodes, la profondeur du GNN et le nombre d'itérations. Ce procédé vise à limiter la difficulté initiale de l'apprentissage, non à établir que le modèle comprend la structure linguistique des symboles. Le meilleur état observé au cours de chaque niveau est sauvegardé pour limiter la dégradation en fin d'entraînement.

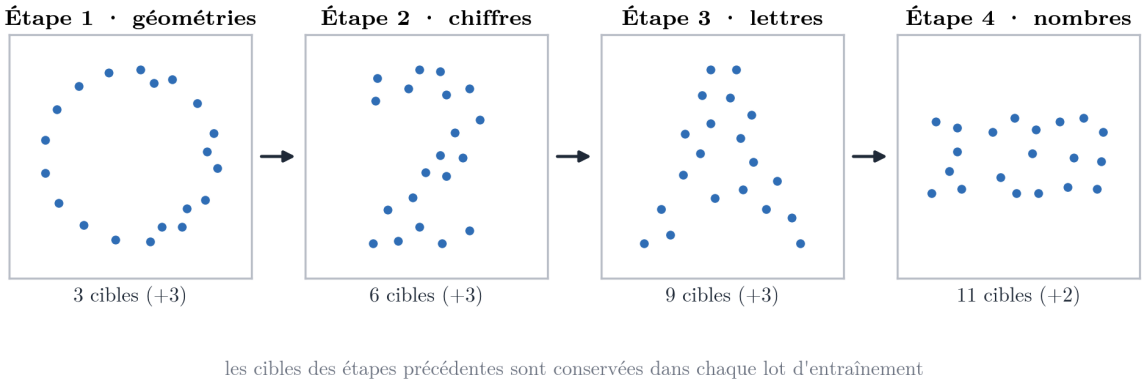


FIGURE 3.6 – Les quatre étapes du curriculum réduit, avec un exemple de cible et le nombre de formes ajoutées à chaque étape. Les cibles des étapes antérieures sont conservées dans le lot d'entraînement.

3.7 Algorithme d'entraînement

L'algorithme 1 résume la boucle réellement implémentée. Elle diffère d'une boucle de renforcement multi-agents en ce qu'aucune récompense individuelle n'est attribuée : une seule perte par épisode, calculée sur l'ensemble de l'essaim, est rétropropagée vers les paramètres partagés. La figure 3.7 montre pourquoi cette rétropropagation est possible : les T pas de simulation forment un graphe de calcul enregistré par PyTorch, reliant les paramètres du GNN à la perte finale à travers chaque pas de mouvement. C'est aussi la source de la contrainte de mémoire principale du prototype, discutée dans la sous-section suivante.

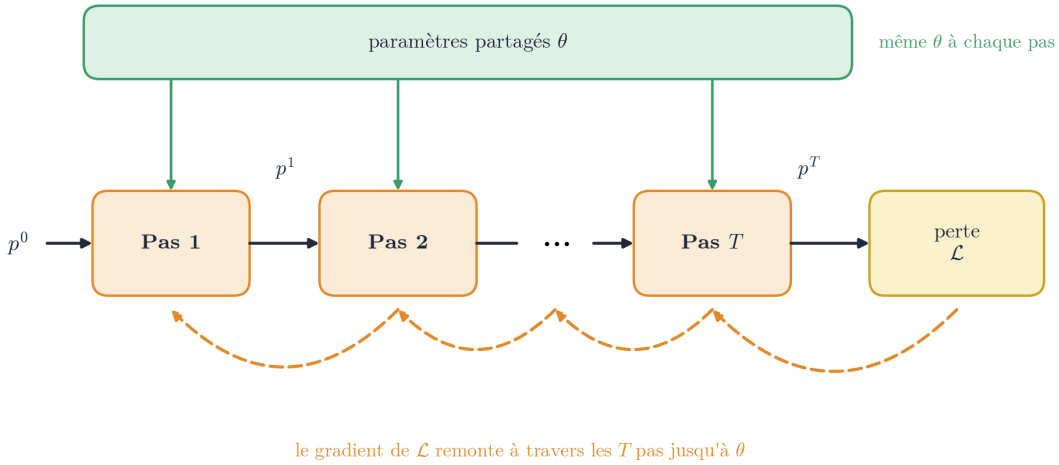


FIGURE 3.7 – Rétropropagation à travers les pas de simulation. Les T pas forment un graphe de calcul enregistré ; le gradient de la perte finale remonte jusqu'aux paramètres partagés du contrôleur à travers toute la trajectoire.

La sauvegarde du meilleur état, et non du dernier, répond à une observation pratique : les dernières itérations d'un niveau de curriculum ne sont pas nécessairement les meilleures pour l'ensemble des cibles, car les paramètres continuent d'évoluer vers la nouvelle combinaison de formes. La sauvegarde du meilleur état réduit cette dérive sans constituer une protection contre l'oubli catastrophique, qui reste un phénomène possible [10, 17].

3.8 Reproductibilité et baselines

Les paramètres des simulations, de l'entraînement et de l'évaluation sont conservés dans des fichiers de configuration versionnés, avec une graine explicite. Les modèles et tableaux produits sont séparés du code source. La première baseline est une règle locale de cohésion–répulsion : elle fournit un point de comparaison interprétable pour le regroupement, mais ne reçoit pas la consigne circulaire. Elle est donc utile comme contrôle de base, et non comme comparaison informationnellement équivalente avec le GNN.

Algorithm 1 Entraînement de la politique partagée

Require: configuration $\mathcal{C}$, graine s, jeu de cibles $\mathcal{D}$
Ensure: $\theta^\star$, courbe des pertes, configuration effective
initialiser θ, l'optimiseur Adam et le meilleur score $b \leftarrow +\infty$
for itération $= 1 \rightarrow I$ **do**
    tirer un lot de B cibles et départs aléatoires
    for chaque épisode du lot **do**
        $t \leftarrow 0$
        while $t < T$ **do**
            construire $\mathcal{G}^t$ à partir des positions courantes
            $u_{1..N} \leftarrow \pi_\theta(o^t_{1..N}, \mathcal{G}^t, Q)$ ▷ politique partagée
            $p^{t+1} \leftarrow p^t + \Delta t\, u$, $\|u^t_i\| \le v_{\max}$
            appliquer les pannes éventuelles et $t \leftarrow t+1$
        end while
        calculer $\mathcal{L}$ du lot (forme, collisions, mouvement, couverture)
    end for
    $\theta \leftarrow \mathrm{Adam}(\theta, \nabla_\theta \mathcal{L})$
    if meilleure perte de la campagne **then**
        sauvegarder $\theta^\star$ et la configuration utilisée
    end if
end for

3.9 Organisation de l'implémentation

La mise en œuvre sépare volontairement les composants qui relèvent de la simulation de ceux qui relèvent de l'apprentissage. Le module de simulation met à jour les positions, applique les limites de l'arène, construit le voisinage et gère les pannes. Le module de génération de formes produit les images et les nuages de points. Le module de modèle contient le contrôleur GNN, tandis que les modules d'entraînement et d'évaluation exécutent les épisodes, calculent les pertes et enregistrent les résultats.

Cette séparation a un intérêt pratique. Une même condition initiale peut être évaluée avec la règle de regroupement ou avec le GNN sans modifier la dynamique de base. Elle permet également de tester séparément le calcul du voisinage, le respect de la vitesse maximale, le traitement d'un robot isolé et l'invariance au réordonnancement des robots. Les figures ne pilotent pas la simulation : elles sont construites après l'exécution à partir des positions enregistrées.

Les paramètres numériques sont regroupés dans des fichiers de configuration. Ils comprennent notamment la graine aléatoire, le nombre de robots, le rayon de communication, la durée d'un épisode, la vitesse maximale et les poids de chaque terme de la perte. Après un entraînement, le projet sauvegarde le modèle, la configuration utilisée, la liste des pertes et les métriques disponibles. Cette traçabilité ne transforme pas une seule graine en résultat général, mais elle permet de rejouer une expérience, de comparer deux réglages et de repérer l'origine d'une différence.

3.10 Restitution des trajectoires

Une petite API locale complète le prototype pour la démonstration. Elle charge le modèle du curriculum, exécute un épisode pour une forme connue et retourne les positions successives des robots au format JSON. Une page Web peut alors dessiner les points de la cible et animer ces positions. L'interface n'exécute pas les règles de contrôle elle-même : le calcul est réalisé par le modèle PyTorch, puis seul le résultat est affiché dans le navigateur.

Ce choix évite de présenter comme résultat du GNN une animation construite avec des règles manuelles en JavaScript. Il ne s'agit pas d'un système de contrôle temps réel ni d'une communication par WebSocket. L'API est un outil de visualisation local, utile pour expliquer le modèle et rejouer un scénario déterminé par une graine.

3.11 Informations locales et information de mission

Le terme local peut recouvrir plusieurs réalités. Dans le contrôleur proposé, les informations provenant d'autres robots sont locales au sens strict : elles sont limitées aux voisins reliés dans le graphe de communication. Pour chaque arête, le message dépend des états des deux robots, de leur décalage relatif et de leur distance. Aucun message n'est envoyé directement par un robot situé hors de portée.

La consigne de forme est d'une autre nature. Elle est définie avant l'exécution de la mission et est partagée par l'essaim. Dans le curriculum réduit, un identifiant de forme est converti en représentation interne par le réseau. Dans la configuration complexe, le contrôleur reçoit en plus un vecteur local vers le point de contour le plus proche. Cet ajout facilite le suivi de formes détaillées, mais donne aussi une information géométrique plus riche. Les deux configurations ne doivent donc pas être confondues lors de l'analyse des résultats.

Cette séparation permet de répondre à une objection fréquente. Le système n'est pas centralisé parce qu'une forme est connue de tous : une carte de mission ou une balise peut être fournie avant le déploiement sans transmettre à chaque pas l'état complet de tous les robots. En revanche, le système ne serait plus strictement local si un serveur recalculait à chaque instant une affectation globale et la distribuait robot par robot. Cette seconde hypothèse n'est pas utilisée dans le simulateur.

3.12 Configurations d'entraînement

La première configuration entraîne vingt robots sur un cercle. Elle emploie une dimension cachée de 32, deux tours de passage de messages, une taille de lot de huit épisodes et 80 itérations. Un épisode comporte vingt pas de simulation. Ce réglage court a été choisi

pour valider la cohérence du pipeline avant de lancer des campagnes plus coûteuses. Le modèle obtenu est conservé avec la courbe de perte et la configuration correspondante.

Le curriculum réduit reprend ensuite un modèle compatible et introduit progressivement trois géométries, trois chiffres, trois lettres et deux nombres. Chaque niveau conserve les cibles précédentes afin de réduire le risque qu'un nouvel apprentissage efface complètement les comportements acquis. Cette stratégie n'élimine pas le problème de l'oubli catastrophique, mais elle évite d'entraîner chaque niveau uniquement sur la dernière forme ajoutée.

Une campagne complexe est également définie. Elle utilise cinquante robots, cinquante points de contour, une dimension cachée de 64, trois tours de messages et des épisodes de soixante pas. Elle est plus proche du niveau de détail requis pour les symboles, mais son coût sur processeur est plus élevé. Au moment de la rédaction, le niveau géométrique de cette campagne a été exécuté ; les niveaux suivants doivent encore être entraînés avant de pouvoir être analysés comme résultats complets.

3.13 Mesures utilisées pendant et après l'apprentissage

La distance de Chamfer est utilisée parce qu'elle compare deux ensembles de points sans imposer d'ordre ou d'affectation fixe. Le premier terme vérifie qu'aucun robot ne s'éloigne fortement de la cible ; le second vérifie qu'aucune partie de la cible ne reste trop loin de tous les robots. Elle n'est pas suffisante à elle seule. Deux configurations visuellement différentes peuvent avoir une distance voisine, notamment lorsqu'une zone fine est mal couverte.

Le projet complète donc cette distance par une pénalité de collision et par une mesure de régularité des distances au plus proche voisin. Le taux de collision décrit le nombre de paires qui deviennent trop proches au cours de l'épisode. La régularité renseigne sur la répartition de l'essaim : une valeur faible indique que les espacements sont plus homogènes, mais ne garantit pas que la forme soit correcte. Enfin, un taux de succès est calculé à partir d'un seuil explicite de distance. Ce seuil doit être affiché avec le résultat, car son choix influence directement l'interprétation du pourcentage de succès.

Pour les lettres et les nombres, une évaluation visuelle reste nécessaire dans cette première version. Le mémoire ne déclare pas qu'un caractère est lisible sur la seule base d'une faible distance de Chamfer. Une suite logique du travail serait de définir un protocole de lisibilité, par exemple avec des évaluateurs humains ou un classifieur séparé entraîné sur des rendus d'essaims.

3.14 Définition opérationnelle des mesures

Les quatre mesures rapportées pendant et après l'entraînement sont définies ici de manière explicite, afin que chaque valeur du chapitre 4 puisse être recalculée à partir des positions enregistrées.

La distance de forme est la distance de Chamfer symétrique de l'équation (3.4), évaluée sur la configuration finale. La valeur rapportée dans les tableaux du chapitre 4 emploie les distances non carrées, tandis que l'équation (3.4), utilisée comme terme d'entraînement, emploie les distances carrées pour lisser le gradient ; les deux versions ne diffèrent que par l'échelle des valeurs, pas par l'ordre des configurations. Le taux de collisions normalise, à chaque pas, le nombre de paires de robots plus proches que la distance de sécurité d_s par le nombre de paires possibles $N(N-1)/2$, puis moyenne cette proportion sur les T pas de l'épisode. Ce taux ne représente pas une collision mécanique : le simulateur ne modélise ni masse, ni contact, ni rebond. Il signale la fréquence des configurations incompatibles avec la marge de sécurité.

La régularité de répartition compare les statistiques — moyenne et écart-type — des distances au plus proche voisin dans l'essaim et dans la cible : leur écart quadratique, noté $\mathcal{E}_{cov}$, forme le terme de régularisation de couverture de l'équation (3.2). Une valeur faible indique que les robots sont espacés de façon plus homogène ; une valeur élevée signale des zones denses et des zones presque vides, fréquentes lorsque l'essaim se replie sur une fraction du contour.

Enfin, le taux de succès et le pas de convergence sont définis à partir d'un seuil unique τ appliqué à la distance de forme :

$$\text{succès} = \mathbb{1}\Big[t_{conv} \leq T-1\Big], \qquad t_{conv} = \min\Big\{t \,:\, d_C(P^t, Q) \leq \tau\Big\}. \tag{3.5}$$

Le succès est donc compté uniquement lorsque le seuil est franchi au moins un pas avant la fin de l'épisode : un franchissement n'intervenant qu'au dernier pas est ignoré, convention héritée du calcul du pas de convergence et illustrée par la graine 103 du tableau 4.3. Le pilote utilise $\tau = 0{,}5$. Ce seuil est volontairement explicite : il est simple à changer, et toute variation du seuil se répercute directement sur les pourcentages de réussite. Les tableaux du chapitre 4 rappellent donc les valeurs avec lesquelles ils ont été produits.

3.15 Déroulement détaillé d'un épisode

Un épisode commence par le choix d'une cible et le tirage des positions initiales dans l'arène. Les vitesses sont initialisées à zéro. À chaque pas, le simulateur calcule toutes les distances inter-robots puis construit le graphe de communication en conservant uniquement les paires séparées par une distance inférieure au rayon fixé. Cette opération est

répétée à chaque instant : un voisin au début de l'épisode peut ne plus être voisin après quelques déplacements.

Le contrôleur traite ensuite l'état de chaque robot. Il encode la vitesse, la consigne disponible et l'identifiant de la cible. Pour chaque arête du graphe, il construit un message à partir des états latents des deux extrémités et de leur position relative. Les messages reçus sont moyennés, l'état latent est mis à jour, puis le même mécanisme est répété selon le nombre de tours configuré. Un décodeur transforme enfin l'état final en une commande de vitesse à deux dimensions. La norme de cette commande est limitée avant la mise à jour de la position.

Pendant l'entraînement, la simulation est différentiable. Les opérations décrites ci-dessus sont réalisées avec des tenseurs PyTorch, ce qui permet de calculer la dérivée de la perte par rapport aux poids du GNN. Après la rétropropagation, l'optimiseur Adam modifie les paramètres. Pendant l'évaluation, les poids sont gelés et le même épisode fournit uniquement des mesures. Cette différence est importante : l'évaluation ne doit pas modifier le modèle après avoir vu les scénarios testés.

Le simulateur limite les positions aux frontières de l'arène. Ce choix empêche des valeurs non physiques de se propager dans la simulation, mais il peut créer un effet de bord : un robot qui atteint une limite ne se comporte pas comme dans un environnement ouvert. Les résultats sont donc interprétés dans une arène carrée finie, non comme une preuve de comportement dans tout plan euclidien.

3.16 Choix des hyperparamètres

Les hyperparamètres ne sont pas présentés comme des constantes universelles. Le rayon de communication définit le compromis entre localité et connectivité. S'il est trop faible, le graphe se fragmente et les robots isolés ne reçoivent plus de messages. S'il est trop élevé, tous les robots deviennent voisins et l'hypothèse de communication locale perd son intérêt. La valeur de 2,5 utilisée dans les premières expériences est un choix de simulation à documenter, pas une distance radio correspondant à une plateforme précise.

La profondeur du GNN règle le nombre de tours de messages disponibles au sein d'un pas de simulation. Deux tours sont employés dans le curriculum réduit. Cette valeur permet à un robot d'intégrer indirectement des informations provenant de voisins de voisins, tout en gardant un modèle léger. La campagne complexe utilise trois tours, car les contours demandent une coordination plus fine. Une profondeur supérieure pourrait étendre l'information disponible, mais son effet doit être mesuré : elle augmente le nombre de calculs et peut réduire la stabilité de l'apprentissage.

La dimension cachée contrôle la capacité du modèle à représenter les états locaux et les messages. La première campagne utilise 32 composantes latentes, ce qui limite le nombre de paramètres et facilite les essais rapides. La campagne complexe passe à 64

composantes. Cette augmentation ne constitue pas une preuve qu'un modèle plus grand est meilleur ; elle répond au besoin de représenter des formes avec davantage de points et des épisodes plus longs. Une étude complète devrait comparer plusieurs dimensions à budget d'entraînement identique.

La vitesse maximale et le pas de temps sont liés. Une vitesse élevée ou un pas trop grand permet aux robots de rejoindre rapidement la cible, mais peut provoquer des dépassements et des collisions. À l'inverse, un pas trop petit rend l'épisode long et peut empêcher la forme de se construire dans la durée choisie. Les pénalités de mouvement et de collision servent à régulariser ce compromis. Les valeurs retenues sont conservées dans les fichiers de configuration afin que chaque résultat puisse être interprété avec son contexte numérique.

3.17 Valeurs numériques retenues

Le tableau 3.2 rassemble les valeurs effectives des trois campagnes. Ces valeurs ne sont pas interprétées comme optimales : elles ont été choisies pour rendre chaque campagne exécutable sur un poste de travail standard et pour stabiliser l'expérimentation.

TABLE 3.2 – Hyperparamètres des trois campagnes d'entraînement, repris des fichiers de configuration versionnés.

Paramètre	Cercle	Curriculum réduit	Formes complexes
Graine	41	43	59
Nombre de robots	20	20	50
Points de la cible	20	20	50
Pas de simulation T	20	20	60
Pas de temps Δt	0,3	0,3	0,2
Itérations	80	4×30	120 à 200
Taille de lot	8	8	4
Taux d'apprentissage	0,003	0,002	0,001
Dimension cachée	32	32	64
Tours de messages	2	2	3
Rayon de communication	2,5	2,5	2,5
Vitesse maximale	1,0	1,0	1,0
Distance de sécurité	0,3	0,3	0,22
Rayon de cible	2,0	2,0	2,8
λ_f (forme)	1,0	1,0	1,0
λ_c (collisions)	1,0	1,0	0,5
λ_u (mouvement)	0,01	0,01	0,005
λ_r (couverture)	0,20	0,20	0,15

Trois différences entre le curriculum réduit et la campagne complexe méritent d'être soulignées. Le nombre de robots et de points est doublé, ce qui multiplie par environ quatre le nombre de paires à considérer pour le voisinage et les collisions. Les épisodes sont trois fois plus longs, ce qui augmente la mémoire nécessaire à la rétropropagation à

travers la simulation. Enfin, le contrôleur reçoit en plus le vecteur vers le contour cible le plus proche, une information géométrique qui n'est pas disponible dans les deux premières campagnes. Les résultats de ces campagnes ne sont donc pas comparables entre eux sans cette précision.

3.18 Gestion des formes et du curriculum

Le jeu de données de formes est séparé du modèle. Il contient des images et les nuages de points associés. Cette séparation a deux avantages. D'une part, la génération ou la correction d'une cible n'oblige pas à modifier l'architecture du GNN. D'autre part, elle permet de définir explicitement quels fichiers appartiennent à l'entraînement, à la validation ou au test. Cette discipline devient indispensable dès que l'on veut déclarer qu'un modèle généralise à des formes non vues.

Le curriculum utilisé dans le projet commence par les géométries simples. Le modèle voit ensuite les chiffres, puis quelques lettres et nombres. Le principe n'est pas que les premières formes soient intrinsèquement plus importantes ; elles fournissent un problème initial plus stable. Un cercle, un carré et un triangle permettent notamment de vérifier que la perte de forme et la pénalité de collision entraînent les mouvements dans une direction utile avant d'ajouter des contours plus irréguliers.

Chaque nouveau niveau conserve les cibles des niveaux précédents. Sans cette précaution, le modèle pourrait optimiser fortement les nouvelles formes et perdre la capacité acquise sur le cercle ou le triangle. Cette conservation réduit le risque d'oubli, mais elle ne le supprime pas. Elle augmente également la diversité des épisodes et peut ralentir l'amélioration sur une cible particulière. Ces compromis sont normaux dans un apprentissage progressif et doivent être visibles dans les courbes et les métriques par forme.

3.19 Architecture logicielle et reproductibilité

Ce chapitre décrit l'organisation du prototype afin de distinguer les choix scientifiques des détails nécessaires à leur reproduction. Le dépôt associe le code source, les configurations et la documentation. Les résultats lourds, tels que les modèles entraînés, les animations et les tableaux bruts, sont produits séparément à partir de ces éléments versionnés.

L'architecture répond à une contrainte principale : une même condition initiale doit pouvoir être rejouée avec un contrôleur différent, sans que la dynamique de simulation soit modifiée par cette substitution. Une seconde contrainte complète la première : aucun résultat présenté dans le mémoire ne doit dépendre d'un réglage caché dans le code. Tous les paramètres numériques sont donc portés par des fichiers de configuration versionnés.

3.19.1 Vue d'ensemble des modules

Le prototype est découpé en modules indépendants qui se limitent à des échanges de données explicites. Le simulateur ne connaît pas la politique qu'on lui fournit ; le modèle ne connaît pas la mécanique de l'arène ; les scripts d'évaluation consomment des fichiers produits par l'entraînement plutôt que des objets Python en mémoire. La figure 3.8 résume ces responsabilités et les flux entre elles.

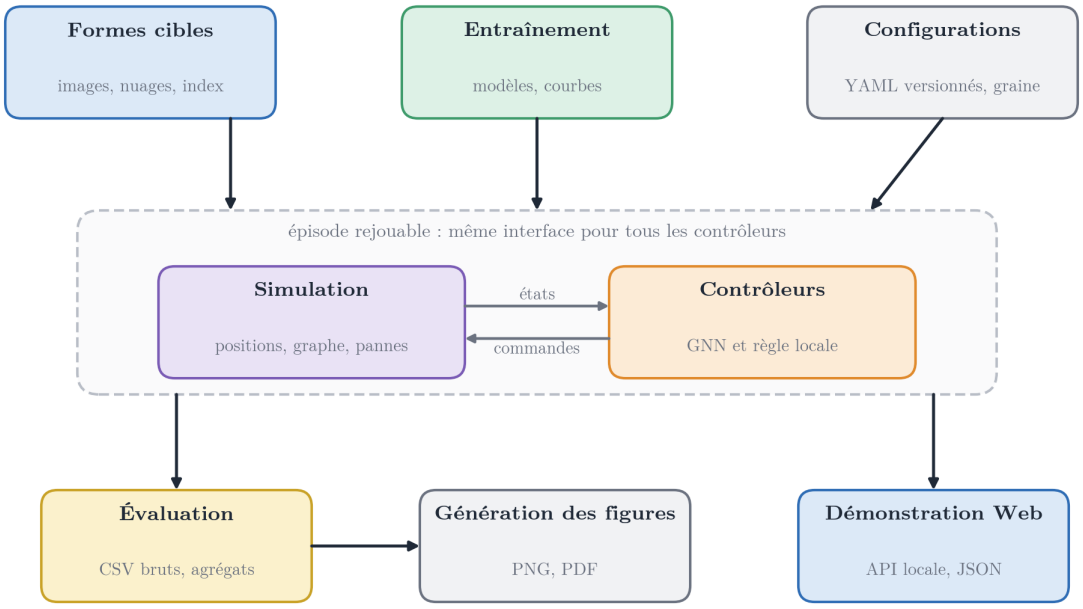


FIGURE 3.8 – Organisation en modules du prototype. Chaque module consomme et produit des artefacts explicites ; la séparation entre simulation, modèle et mesure permet d'évaluer plusieurs contrôleurs sur les mêmes conditions initiales.

Le tableau 3.3 détaille le rôle de chaque module et les artefacts qu'il produit. Cette séparation a trois effets pratiques. Elle permet de tester le voisinage, le respect de la vitesse maximale ou le traitement d'un robot isolé sans exécuter d'entraînement. Elle empêche une fonction de visualisation d'influencer la dynamique simulée. Elle rend enfin possible la comparaison directe d'une règle écrite à la main et du contrôleur appris, puisque les deux sont appelés à travers la même interface d'épisode.

3.19.2 Structure du dépôt

Le projet regroupe les sources, les configurations, les données de formes, la documentation, les tests et les sorties d'expériences dans des répertoires distincts, dont les responsabilités sont celles du tableau 3.3. Les entrées de données générées ne sont pas versionnées ; seuls les résumés et la documentation le sont, ce qui évite de conserver des binaires tout en gardant la possibilité de les régénérer à partir des configurations.

TABLE 3.3 – Modules du prototype, responsabilités et artefacts produits.

Module	Responsabilité	Artefacts principaux
Simulation	dynamique, voisinage, arène, pannes, animation	positions, graphe, animation
Formes cibles	rendu et échantillonnage des formes	images, tableaux de points, index
Contrôleurs	contrôleur GNN et contrôleurs de référence	commandes, vérification de vitesse
Entraînement	boucle d'entraînement, pertes, curriculum	modèles, courbes, métriques
Évaluation	scénarios de pannes et de passage à l'échelle	CSV bruts, agrégats, figures
Démonstration Web	API locale de visualisation	réponses JSON de trajectoires
Génération des figures	figures du mémoire	PNG des schémas

Chaque campagne d'entraînement écrit ses sorties dans un répertoire qui porte le nom de l'expérience et le niveau du curriculum concerné. Ainsi, les niveaux *geometry*, *digits*, *symbols* et *numbers* du curriculum réduit coexistent sans se remplacer mutuellement, ce qui permet de comparer les pertes d'un niveau à l'autre après coup.

3.19.3 Cycle d'exécution d'un épisode

Un épisode commence par le tirage de positions initiales et par le choix d'une forme cible. À chaque pas, le simulateur construit le graphe de communication à partir des positions courantes. Chaque robot actif encode alors son observation locale, agrège les messages de ses voisins et produit une vitesse bornée. Le simulateur applique simultanément les commandes, puis la boucle recommence. Après le dernier pas, la configuration finale est comparée au nuage de points de la cible. La figure 3.9 résume ce cycle, identique pendant l'entraînement et l'évaluation.

Pendant l'entraînement, cette boucle est différentiable et la perte remonte jusqu'aux paramètres du GNN. Pendant l'évaluation, les paramètres sont gelés et le même mécanisme ne produit que les métriques. Cette séparation réduit le risque d'évaluer un modèle dans un mode différent de celui utilisé pour son entraînement.

Un point d'implémentation mérite d'être précisé. Le calcul du voisinage est reconstruit à chaque pas, et non mis à jour de façon incrémentale. Cette approche est plus coûteuse, mais elle supprime une classe entière d'erreurs : aucune arête périmée ne peut survivre à un déplacement, et la matrice d'adjacence utilisée par le contrôleur est toujours cohérente avec les positions. Les tests unitaires portent d'ailleurs sur cette cohérence, et non sur un détail de représentation.

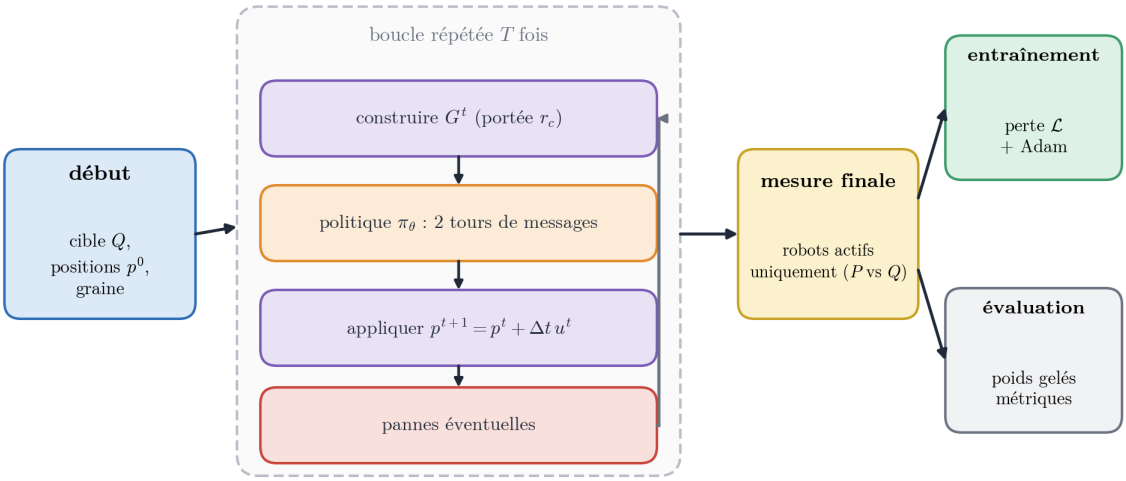


FIGURE 3.9 – Cycle de vie d'un épisode. La boucle centrale est répétée T fois ; la mesure finale ne porte que sur les robots actifs et alimente la perte (entraînement) ou les métriques (évaluation).

3.19.4 Configuration et traçabilité

Les paramètres numériques ne sont pas dispersés dans le code : ils sont regroupés dans des fichiers de configuration. Le tableau 3.4 liste les configurations du projet et leur objet.

TABLE 3.4 – Fichiers de configuration du projet et objet de chacun.

Fichier	Objet	Valeurs notables
`shapes.yaml`	jeu de formes standard	image 128×128, 20 points
`shapes-complex.yaml`	jeu de formes complexes	50 points par forme
`simulator.yaml`	démonstration de la règle locale	portée, vitesse maximale
`training-circle.yaml`	entraînement sur le cercle	20 robots, 80 itérations, graine 41
`curriculum.yaml`	curriculum réduit	4 niveaux de 30 itérations
`curriculum-complex.yaml`	campagne complexe	50 robots, 60 pas, 3 tours
`curriculum-complex-smoke.yaml`	vérification rapide	pipeline uniquement
`evaluation-pilot.yaml`	protocole pilote	10 épisodes, 4 retraits, 3 tailles

Chaque exécution sauvegarde, à côté des résultats, la configuration réellement utilisée. Un répertoire de campagne contient typiquement le modèle final, le meilleur modèle observé, la configuration effective, la liste des pertes itération par itération, la courbe correspondante, les métriques par cible et la liste des cibles vues. Cette traçabilité ne transforme pas une seule graine en résultat général ; elle permet en revanche de rejouer

un épisode, de comparer deux réglages et d'identifier l'origine d'une différence lorsque le code évolue.

La graine aléatoire est le pivot de cette reproductibilité. Elle conditionne le tirage des positions initiales, l'ordre des cibles présentées au contrôleur et les scénarios de retrait. Une même graine produit donc le même épisode tant que l'implémentation ne change pas ; une graine différente produit un scénario indépendant. Le passage d'une reproductibilité individuelle à une conclusion statistique suppose de cumuler plusieurs graines, ce qui fait l'objet du protocole d'évaluation du chapitre 4.

3.19.5 Tests et contrôles de non-régression

La confiance dans les résultats numériques repose sur une suite de tests unitaires qui vérifie des propriétés, et non des valeurs obtenues après coup. Le tableau 3.5 résume les propriétés contrôlées par module.

TABLE 3.5 – Propriétés vérifiées par la suite de tests unitaires, par composant.

Composant testé	Propriétés vérifiées
Simulation	respect du rayon de communication et de l'état actif, bornure de la vitesse, robot en panne immobile et non compté en collision, reproductibilité d'une graine, validité d'une règle locale pour un robot isolé
Contrôleur GNN	tailles d'essaim et voisinages variables, invariance par permutation des robots, invariance par translation globale, conditionnement par la consigne, traitement des robots isolés et communication désactivée
Métriques	valeurs exactes sur des cas simples, détection d'une paire en collision
Entraînement	pertes proches de zéro sur des cas attendus, absence d'auto-boucle dans l'adjacence, cohérence de la cible, entrée de plusieurs conditions de cible
Génération des formes	normalisation et reproductibilité des points, présence de l'index et de l'archive

Deux de ces propriétés méritent d'être commentées. L'invariance par permutation garantit que le réordonnancement des robots dans une liste, simple artefact d'implémentation, ne change pas les commandes produites : c'est la contrepartie testée de la moyenne des messages. L'invariance par translation vérifie que le contrôleur ne dépend pas du repère absolu, mais des relations entre robots et de la consigne. Ces deux propriétés sont des conditions nécessaires de la décentralisation revendiquée ; elles ne garantissent pas la qualité de la formation, mais elles éliminent les échecs qui proviendraient de la représentation des données plutôt que du contrôle lui-même.

3.19.6 Exposition locale de la démonstration

Une petite API locale complète le prototype pour la démonstration. Elle charge le modèle du dernier niveau du curriculum, exécute un épisode pour une forme connue et retourne les positions successives des robots au format JSON. Une page Web peut alors dessiner les points de la cible et animer ces positions. Le tableau 3.6 décrit les routes.

TABLE 3.6 – Routes de l'API locale de visualisation.

Route	Objet	Réponse
`GET /api/health`	disponibilité du modèle	statut et chemin du modèle chargé
`GET /api/targets`	cibles connues	identifiants et fichiers sources
`GET /api/simulate`	exécution d'un épisode	cible, modèle employé, positions par pas

L'interface n'exécute pas les règles de contrôle elle-même : le calcul est réalisé par le modèle PyTorch côté serveur, puis seul le résultat est affiché dans le navigateur. Ce choix évite de présenter comme résultat du GNN une animation construite avec des règles manuelles côté client. Le serveur n'emploie ni accès à Internet, ni WebSocket, ni base de données, et n'écoute que sur la boucle locale ; il s'agit d'un outil d'inspection et d'explication, non d'un système de contrôle temps réel.

Le contrat détaillé des réponses JSON est documenté dans la documentation du projet, ce qui permet de réécrire l'interface d'affichage sans toucher au modèle. La reproductibilité d'une démonstration repose alors sur le couple (forme, graine) : deux personnes lançant la même requête obtiennent exactement les mêmes positions.

3.19.7 Coût de calcul et choix de mise en œuvre

Le prototype utilise PyTorch pour le contrôleur et pour la rétropropagation à travers la simulation, sans dépendance à une bibliothèque de réseaux de graphes : l'échange de messages est implémenté directement, ce qui rend les équations du chapitre 3 lisibles dans le code et évite une couche d'abstraction qui masquerait l'agrégation.

Les épisodes d'un lot sont traités ensemble sous forme de tenseurs, et la rétropropagation conserve l'historique des T pas de simulation. La consommation de mémoire croît donc avec la longueur des épisodes, avec le nombre de robots et avec le nombre de paires de voisins. C'est cette croissance qui motive le profil de vérification rapide de la campagne complexe : il sert à contrôler le pipeline en quelques minutes, sans permettre aucune conclusion sur la qualité des formes. Les campagnes complètes s'exécutent sur processeur en plusieurs dizaines de minutes, ce qui rend la sauvegarde du meilleur état et la reproductibilité par graine d'autant plus nécessaires.

3.19.8 Visualisation et inspection

Les figures de formation ne sont pas seulement des éléments de présentation. Elles servent à détecter des comportements qui peuvent échapper à une moyenne numérique : amas local, points de cible non couverts, trajectoires instables ou collisions récurrentes. Les images de fin d'épisode et les animations sont donc générées à partir du même simulateur que celui utilisé pour les métriques.

L'approche conserve toutefois une frontière nette entre illustration et preuve. Une animation convaincante pour une graine ne remplace pas une évaluation multi-graines ; inversement, une moyenne faible doit être confrontée à la distribution spatiale réellement obtenue. Le chapitre 4 applique ce principe aux résultats disponibles.

Chapitre 4

Expériences et résultats

4.1 Protocole pilote

Les expériences sont organisées en deux phases. La première contrôle que le GNN peut être entraîné sur un cercle et qu'il produit visuellement une distribution autour de la cible. La seconde est un protocole pilote d'évaluation, conçu pour vérifier la chaîne de mesure avant une campagne plus large. Ce pilote fixe dix graines par condition, soit 240 épisodes. Il évalue une cible circulaire avec 20, 40 et 100 robots, ainsi que des retraits de 5, 10 et 20% des robots. Chaque épisode dure 20 pas.

Les métriques rapportées sont la distance de Chamfer, le nombre ou le taux de collisions, la régularité des distances entre voisins et un taux de succès associé à un seuil de distance de Chamfer de 0,5. La distance de Chamfer mesure l'adéquation spatiale à la forme, mais elle ne garantit ni une couverture uniforme ni la lisibilité d'un symbole. Les métriques sont donc complémentaires et doivent être interprétées avec les visualisations finales.

Le protocole distingue deux types de comparaison. La comparaison entre le GNN et la règle de regroupement vérifie que le contrôleur entraîné atteint mieux la cible circulaire dans le cadre retenu. Les variations de taille et de pannes évaluent ensuite le même GNN dans des conditions différentes de celles de l'entraînement. Le protocole ne permet pas encore de séparer l'effet de tous les choix de conception : il manque en particulier une baseline apprise sans communication et une règle manuelle recevant la même consigne de forme.

Avant toute agrégation, chaque épisode est reproductible à partir d'une graine. Les positions initiales sont tirées aléatoirement dans l'arène, puis la même dynamique est appliquée pendant vingt pas. Pour un scénario de panne, le retrait se produit à mi-épisode. Les fichiers bruts conservent une ligne par épisode ; le tableau de synthèse rapporte ensuite la moyenne et l'écart-type de la distance de forme. Cette organisation permet de retrouver les cas individuels qui expliquent une moyenne inhabituelle.

4.2 Résultats préliminaires

Pour l'entraînement initial sur un cercle, la perte observée avec la graine 41 passe de 10,08 à 0,29 en 80 itérations. Cette diminution indique que l'optimisation est fonctionnelle dans le simulateur ; elle ne constitue pas une mesure de généralisation, car elle concerne une configuration et une graine particulières. Les figures 4.1 et 4.3 proviennent de cet entraînement dédié au seul cercle ; le protocole pilote évalue lui le modèle issu du curriculum réduit, comme le précise le chapitre 5.

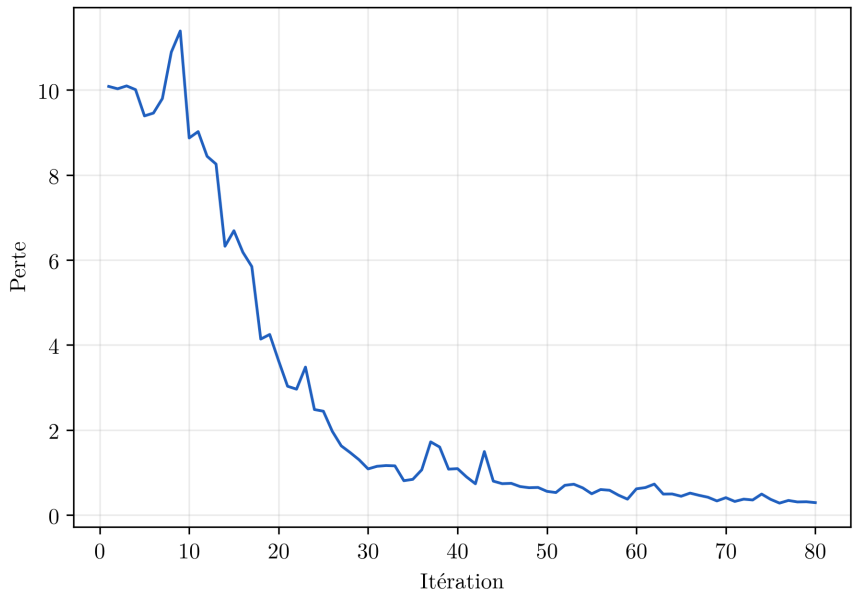


FIGURE 4.1 – Évolution de la perte pendant l'entraînement initial du cercle, pour la graine 41. Cette courbe sert de diagnostic d'optimisation et non d'intervalle de confiance.

Le tableau 4.1 présente les résultats du pilote pour 20 robots. Sans panne, le GNN atteint une distance de Chamfer moyenne de 0,592, contre 3,956 pour la règle de regroupement. Après retrait de 20% des robots, les valeurs sont respectivement 0,638 et 4,162. Dans les mêmes conditions, le taux de collisions mesuré est de 0,021 pour le GNN sans panne et de 0,013 après retrait de 20% des robots ; la règle de regroupement présente des valeurs de 0,187 et 0,129.

TABLE 4.1 – Résultats du protocole pilote à 20 robots (10 graines par condition).

Contrôleur	Pannes	Chamfer moyen	Taux de collisions	Succès
GNN	0%	0,592	0,021	0,10
GNN	20%	0,638	0,013	0,00
Regroupement	0%	3,956	0,187	0,00
Regroupement	20%	4,162	0,129	0,00

Cette comparaison doit être interprétée avec prudence. La règle de regroupement ne reçoit pas de consigne explicite de cercle : elle répond donc à un problème d'information

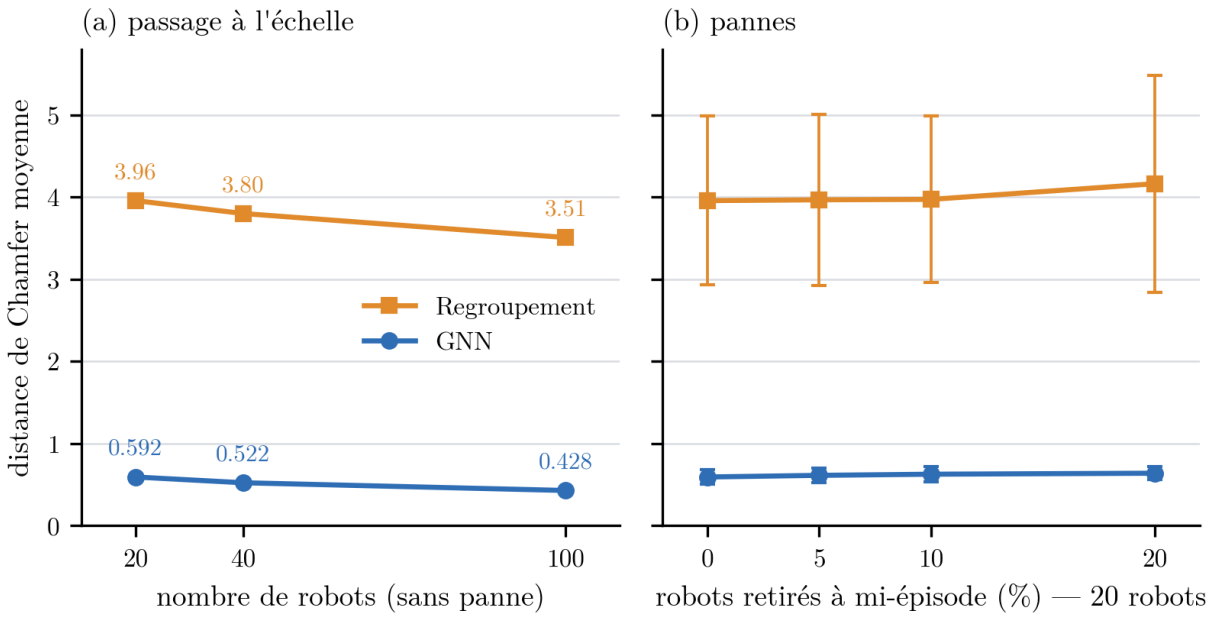


FIGURE 4.2 – Synthèse du protocole pilote calculée à partir de 240 épisodes. À gauche, l'effet de la taille de l'essaim sans panne ; à droite, l'effet des pannes avec 20 robots. Une valeur plus faible de distance de Chamfer indique une meilleure adéquation à la cible circulaire.

différent. Le tableau montre que le contrôleur entraîné est plus proche de la forme circulaire dans ce protocole, mais ne démontre pas qu'il est supérieur à toute règle décentralisée disposant de la même consigne.

4.3 Effet du nombre de robots

Le même modèle est exécuté avec 20, 40 et 100 robots, alors que son entraînement de référence a été réalisé avec 20 robots. Cette expérience vérifie d'abord une propriété pratique : la politique peut être évaluée avec une taille différente sans modifier le nombre de paramètres. Elle vérifie ensuite une propriété empirique : la qualité obtenue reste-t-elle acceptable lorsque le graphe de communication devient plus dense et que la cible doit être couverte par davantage de robots ?

Dans le pilote sans panne, la distance moyenne du GNN est de 0,592 avec 20 robots, de 0,522 avec 40 robots et de 0,428 avec 100 robots. La règle de regroupement reste nettement plus éloignée de la cible, avec des distances respectives de 3,956, 3,799 et 3,507. La diminution observée pour le GNN peut venir d'une meilleure couverture de l'échantillonnage circulaire lorsque davantage de points sont disponibles. Elle ne signifie pas nécessairement que la politique est meilleure dans tous les régimes de densité : dans ce protocole, le nuage cible de vingt points est simplement dupliqué pour s'adapter au nombre

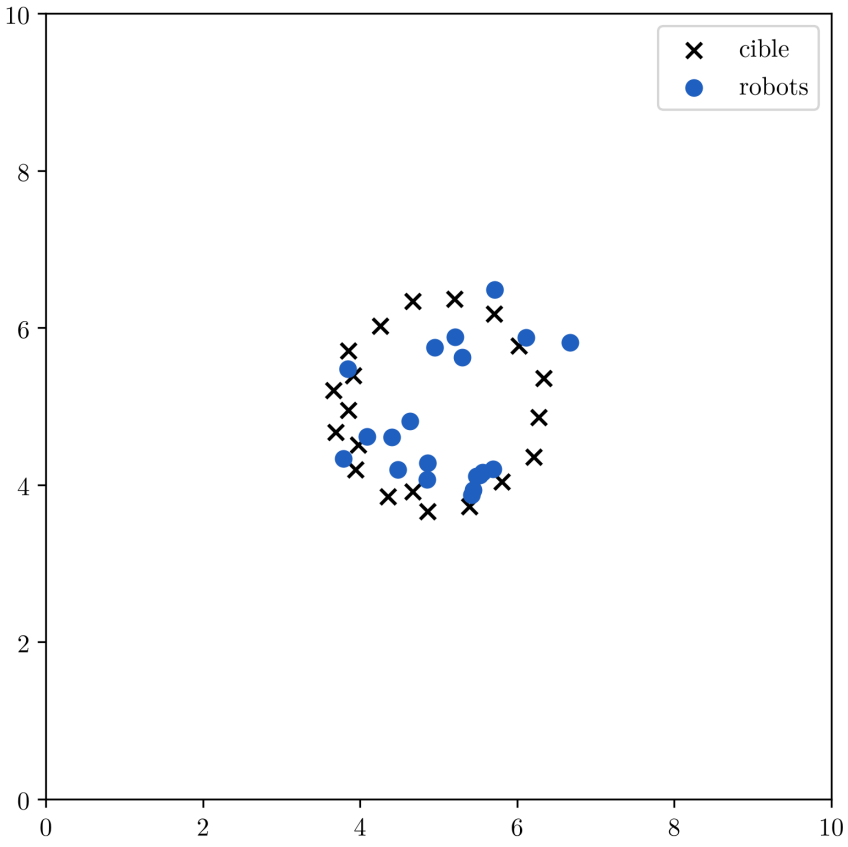


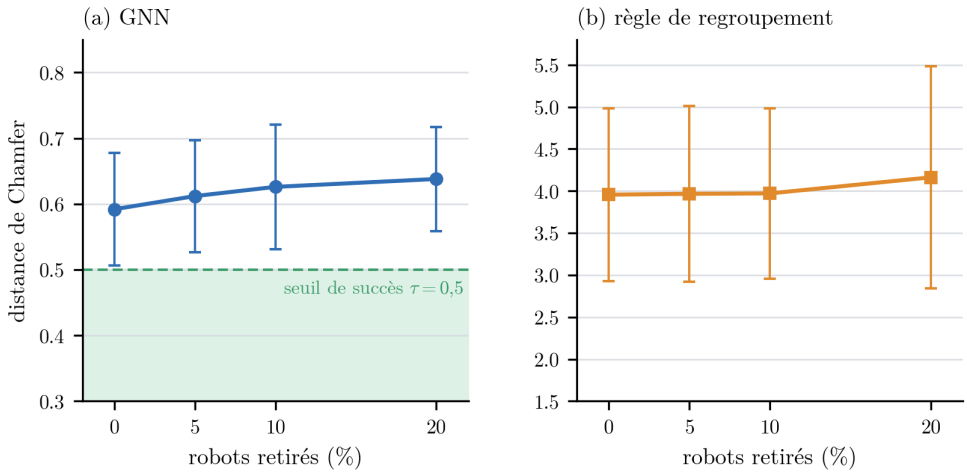
FIGURE 4.3 – Configuration finale obtenue pour l'entraînement du cercle. Les robots représentent l'approximation discrète de la cible par l'essaim.

de robots, ce qui améliore mécaniquement la distance mesurée lorsque la population croît, chaque point cible disposant alors de davantage de robots proches. Le taux de succès, comparé à un seuil fixe alors que l'échantillon cible grossit, subit le même artefact : son passage de 10 à 100 % entre 20 et 100 robots ne doit pas être lu comme un gain de politique. Enfin, le protocole reste limité à dix épisodes par condition.

Le tableau 4.2 récapitule ces résultats. Il doit être lu comme un contrôle de bon fonctionnement à taille variable. Une validation plus ambitieuse demanderait de répéter l'entraînement avec plusieurs graines, d'étudier plusieurs rayons de communication et de vérifier la conservation de la connectivité pendant toute la trajectoire.

TABLE 4.2 – Passage à l'échelle sans panne dans le protocole pilote, avec dix épisodes par condition.

Contrôleur	Robots	Chamfer moyen	Taux de succès
GNN	20	0,592	0,10
GNN	40	0,522	0,30
GNN	100	0,428	1,00
Regroupement	20	3,956	0,00
Regroupement	40	3,799	0,00
Regroupement	100	3,507	0,00



moyenne ± 1 écart-type sur 10 épisodes ; échelles verticales différentes entre (a) et (b)

FIGURE 4.4 – Distance moyenne de Chamfer (moyenne ± 1 écart-type sur dix épisodes) selon la fraction de robots retirés à mi-épisode, pour 20 robots : à gauche, le GNN ; à droite, la règle de regroupement. Le seuil de succès $\tau = 0{,}5$ est indiqué pour le GNN.

La figure 4.4 illustre l'évolution de la distance avec le taux de pannes. Le faible écart observé entre 0% et 20% de pannes pour le GNN est une indication encourageante dans ce cadre précis, mais la taille de l'échantillon ne permet pas d'en déduire un intervalle de confiance robuste. Une campagne finale devra employer au moins 50 répétitions par condition et rapporter la variabilité entre graines.

4.4 Lecture des scénarios de panne

Le protocole de retrait est explicite, et la figure 4.5 en rappelle la chronologie : les robots retirés à mi-épisode sont figés à leur position courante, leurs commandes sont annulées, et ils sont exclus du graphe de communication des robots actifs, du comptage des collisions et de la mesure finale. Ce choix de protocole a deux conséquences qu'il faut garder à l'esprit lors de l'interprétation des résultats du tableau 4.4. D'une part, un robot retiré reste physiquement présent dans l'arène mais n'interagit plus avec les actifs : le protocole modélise une déconnexion propre du réseau, non un robot devenu obstacle ou produisant des mesures aberrantes. D'autre part, la normalisation du taux de collisions par le nombre de paires rend ce dernier directement comparable entre conditions, au prix d'un effet de bord mentionné plus loin.

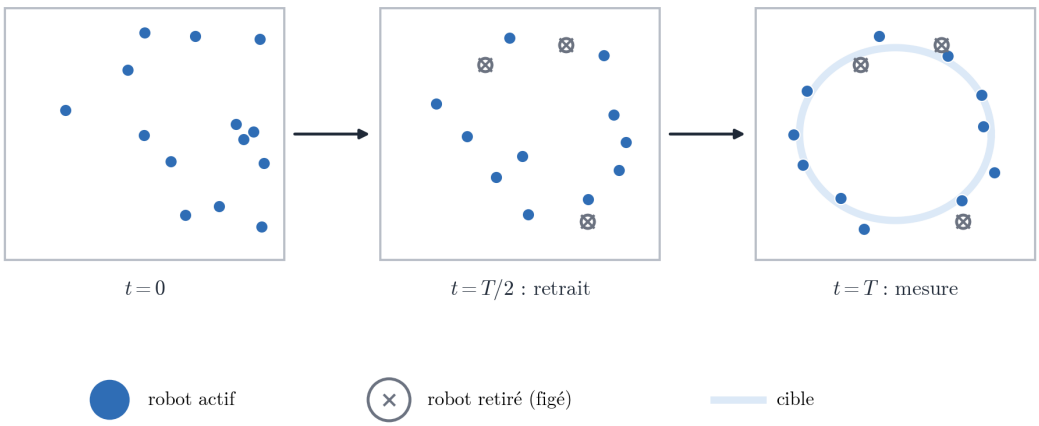
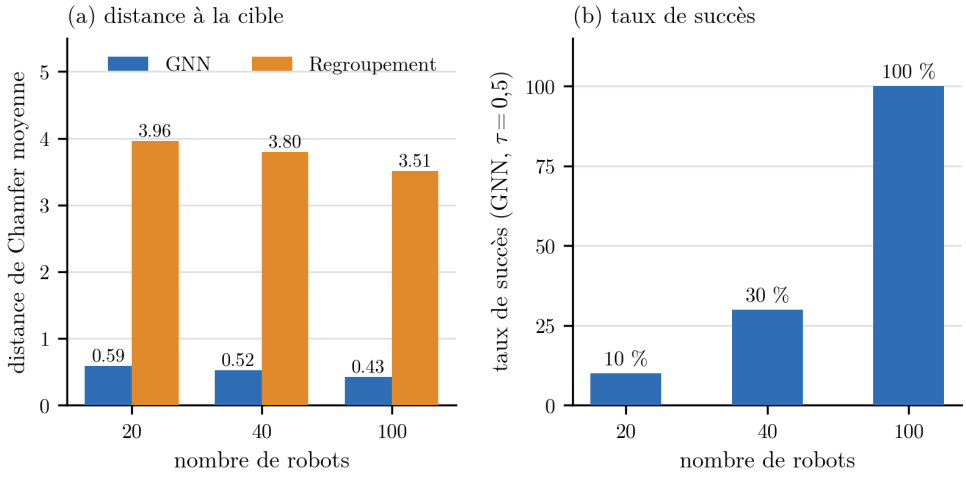


FIGURE 4.5 – Chronologie d'un épisode avec retrait à mi-parcours. Les robots retirés (gris) sont figés et exclus du graphe de communication ; la mesure finale porte sur les robots actifs restants.

Dans le simulateur, une panne est introduite à mi-parcours : une fraction des robots actifs est retirée aléatoirement. Les robots retirés ne contribuent plus aux messages et ne sont plus pris en compte dans la mesure finale de la forme. Cette définition correspond à une disparition propre d'agents du réseau. Elle permet d'étudier comment le reste de l'essaim se réorganise, mais elle est moins sévère qu'une panne matérielle réelle, où un robot peut rester immobile, produire des mesures incohérentes ou gêner physiquement ses voisins.

Le résultat doit donc être formulé avec précision. Le GNN conserve, dans le pilote, une distance de Chamfer plus faible que la baseline après le retrait de 5, 10 ou 20% des robots. Cela montre que la politique n'est pas limitée à une composition rigide de vingt agents. Il ne s'agit pas encore d'une démonstration d'auto-réparation générale. Le protocole n'observe ni le temps nécessaire au retour vers la forme, ni la connectivité du

graphe, ni le comportement d'un robot défaillant qui resterait dans l'arène.



10 épisodes par condition, sans panne ; le taux de succès de la règle de regroupement est nul partout

FIGURE 4.6 – Distance moyenne de Chamfer lors du passage à l'échelle dans le protocole pilote. Cette figure doit être complétée par les valeurs numériques et leur dispersion dans la campagne finale.

4.5 Dispersion des épisodes et atteinte du seuil

Les agrégats masquent une information utile : la dispersion entre graines. Le tableau 4.3 présente, épisode par épisode, les dix départs du pilote à 20 robots sans retrait, triés par distance de forme croissante. Trois observations en sont tirées.

TABLE 4.3 – Épisodes du pilote à 20 robots sans retrait, contrôleur GNN, triés par distance de forme. Le taux de collisions est normalisé par le nombre de paires et la durée de l'épisode.

Graine	Chamfer	Taux de collisions	Pas de convergence	Succès
103	0,499	0,017	20	non
101	0,503	0,020	20	non
107	0,521	0,008	18	oui
109	0,528	0,019	20	non
108	0,561	0,022	20	non
110	0,592	0,008	20	non
105	0,608	0,027	20	non
106	0,643	0,032	20	non
102	0,724	0,033	20	non
104	0,735	0,025	20	non

La première observation est l'amplitude de la dispersion : entre le meilleur et le moins bon épisode, la distance de forme varie d'environ 47 %. La figure 4.8 donne une représentation visuelle de cet étalement, croisée avec le franchissement du seuil. Cette variation

provient uniquement des positions initiales, puisque la graine d'entraînement et le modèle sont fixes. La deuxième observation est que les épisodes les plus proches de la cible ne sont pas systématiquement ceux qui limitent le plus les rapprochements : le meilleur épisode de la liste n'est pas celui qui présente le moins de collisions. La distance de forme et la sécurité ne sont donc pas des dimensions redondantes, ce qui justifie de les rapporter séparément.

La troisième observation concerne le seuil de succès. Avec $\tau = 0{,}5$, un seul épisode sur dix franchit le seuil avant le dernier pas, alors que la distance moyenne (0,592) est du même ordre. La graine 103 illustre la convention retenue : sa distance finale (0,499) est pourtant sous le seuil, mais son franchissement n'intervient qu'au dernier pas et n'est donc pas comptabilisé (équation (3.5)). Le succès mesure ainsi une formation stabilisée avant la fin de l'épisode, non une distance finale favorable. Le taux de succès est ainsi une métrique à forte sensibilité locale : un décalage faible de la distribution des distances peut le faire passer de 10 % à 0 %, sans changement qualitatif de comportement. C'est pourquoi les tableaux du présent chapitre affichent toujours la distance de forme et son écart-type à côté du pourcentage.

Le pas de convergence apporte un complément de lecture. La figure 4.7 donne une visualisation schématique de cette mesure : la distance de forme décroît au fil des pas, et le pas de convergence est le premier instant où elle franchit le seuil τ. Dans la condition du tableau 4.3, le seuil n'est franchi avant la fin de l'épisode que dans un seul cas ; pour les autres départs, la formation est encore en mouvement à vingt pas. Deux interprétations restent ouvertes : la durée de l'épisode est insuffisante, ou la politique ralentit insuffisamment en approchant du contour. Les deux se distinguent en prolongeant l'épisode, ce qui fait partie du protocole prévu.

4.6 Effet des retraits sur les mesures

Le tableau 4.4 détaille l'effet des fractions de retrait à 20 robots. Les distances de forme du GNN passent de 0,592 à 0,638 entre 0 et 20 % de retrait, soit une augmentation d'environ 8 %. La règle de regroupement présente une variation du même ordre de grandeur en valeur absolue, mais à un niveau de distance six fois supérieur.

Table 4.4 – Distance de forme moyenne et écart-type (entre parenthèses) pour 20 robots selon la fraction de robots retirés à mi-épisode.

Contrôleur	0 %	5 %	10 %	20 %
GNN	0,592 (0,086)	0,612 (0,085)	0,626 (0,095)	0,638 (0,079)
Regroupement	3,956 (1,027)	3,966 (1,043)	3,971 (1,014)	4,162 (1,321)

Deux effets de bord apparaissent dans ce tableau. Le premier est que le taux de

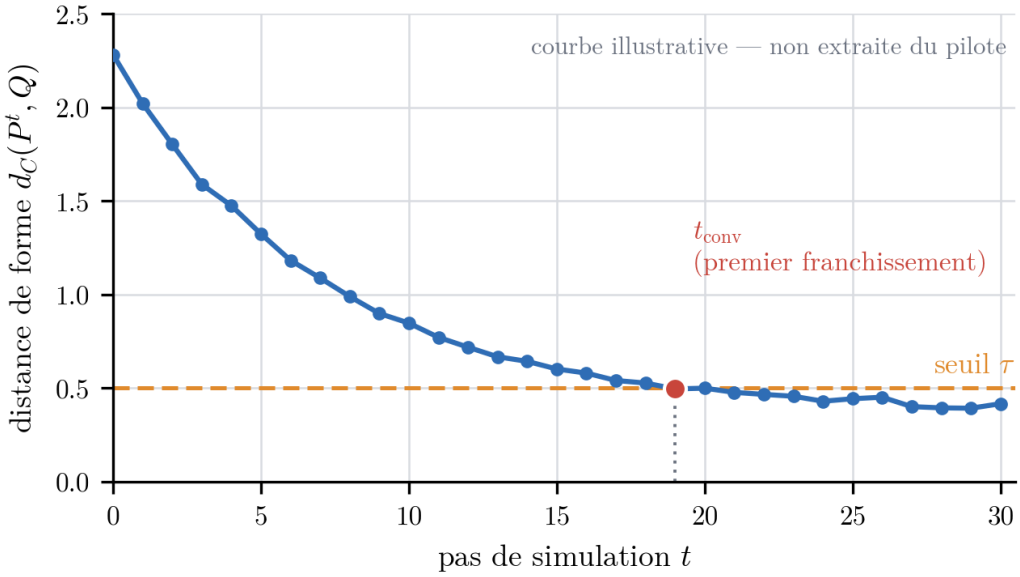


FIGURE 4.7 – Schéma de la mesure de convergence : la distance de forme (bleue) est comparée au seuil τ (orange) à chaque pas ; le premier franchissement définit le pas de convergence t_{conv}. La courbe est illustrative et n'est pas extraite des épisodes du pilote.

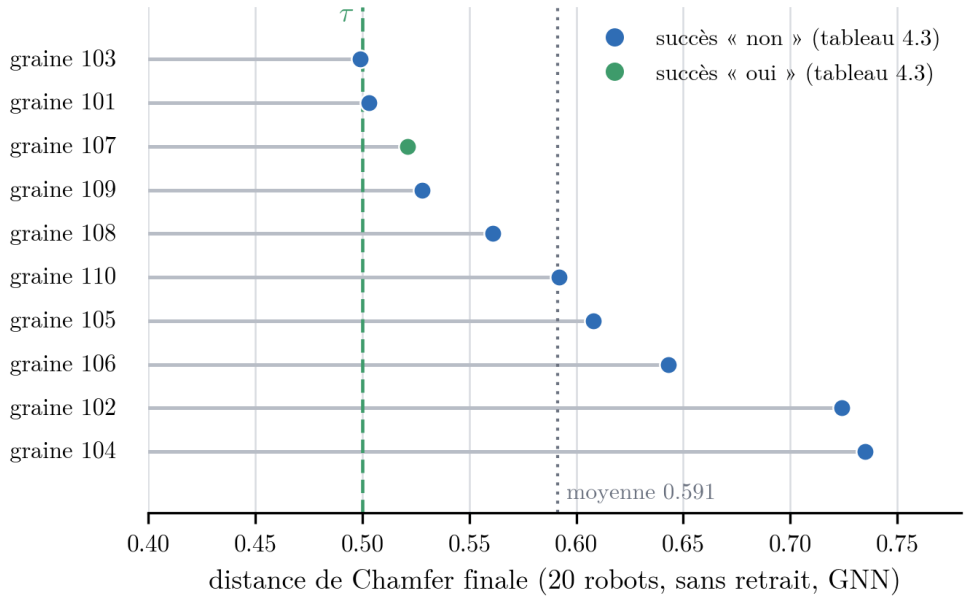


FIGURE 4.8 – Distance de Chamfer finale des dix graines du pilote à 20 robots sans retrait, triées par distance croissante. La ligne verte marque le seuil de succès $\tau = 0{,}5$ et la ligne pointillée la moyenne ; le coloris indique le franchissement du seuil rapporté au tableau 4.3.

collisions du GNN diminue lorsque le nombre de robots retirés augmente : il mesure le ratio de paires trop proches, et la disparition d'agents actifs réduit mécaniquement le nombre de paires concernées. Une baisse du taux de collision n'est donc pas, dans ce protocole, un signe de comportement plus sûr. Le second effet est la croissance de l'écart-type du regroupement lorsque les retraits augmentent : la règle manuelle devient non seulement moins précise, mais aussi plus variable d'un départ à l'autre, ce qui est cohérent avec un comportement de cohésion dont l'issue dépend fortement de la configuration initiale.

4.7 Curriculum de formes

Le curriculum réduit entraîne successivement les niveaux géométrie, chiffres, symboles et nombres. Les pertes finales enregistrées sont respectivement 0,362, 0,251, 0,240 et 0,117. Cette baisse ne doit pas être comparée directement entre niveaux, car les ensembles de cibles et l'état initial du modèle évoluent. Elle confirme surtout que la chaîne d'entraînement peut poursuivre l'optimisation lorsque de nouvelles formes sont ajoutées.

Ces valeurs sont la perte du lot telle qu'enregistrée à la dernière itération de chaque étape. Le tableau 4.5 rapporte une grandeur distincte : la perte réévaluée cible par cible à la fin de chaque étape, puis moyennée entre cibles déjà vues et cibles nouvellement introduites. Les deux séries ne sont donc pas attendues identiques ; elles se complètent sans se contredire. La figure 4.9 donne une lecture visuelle de ces valeurs.

TABLE 4.5 – Perte par cible à la fin de chaque étape du curriculum réduit, séparée entre cibles déjà vues et cibles introduites par l'étape.

Étape	Nombre de cibles	Pertes anciennes	Nouvelles pertes
Géométrie	3	—	0,316
Chiffres	6	0,333	0,126
Symboles	9	0,216	0,141
Nombres	11	0,163	0,117

Deux lectures se dégagent. D'une part, les cibles anciennes ne se dégradent pas au fil des étapes : leur perte moyenne décroît, de 0,333 à 0,163 entre les deuxième et quatrième étapes. La conservation des cibles précédentes dans chaque lot produit donc l'effet recherché, sans garantie théorique contre l'oubli. D'autre part, les nouvelles cibles sont atteintes avec des pertes nettement inférieures à celles des formes géométriques. Ce résultat ne signifie pas que les chiffres et lettres sont *plus faciles* à former : leur contour est plus court et le nombre de points de référence est identique, ce qui rapproche mécaniquement les deux ensembles. La comparaison des pertes entre catégories de formes n'est donc pas interprétable sans normaliser par l'échelle de la cible.

Les courbes de la figure 4.10 complètent le tableau. La variation est plus marquée en début d'étape, lorsque le lot intègre de nouvelles cibles, puis se stabilise. Cette dynamique

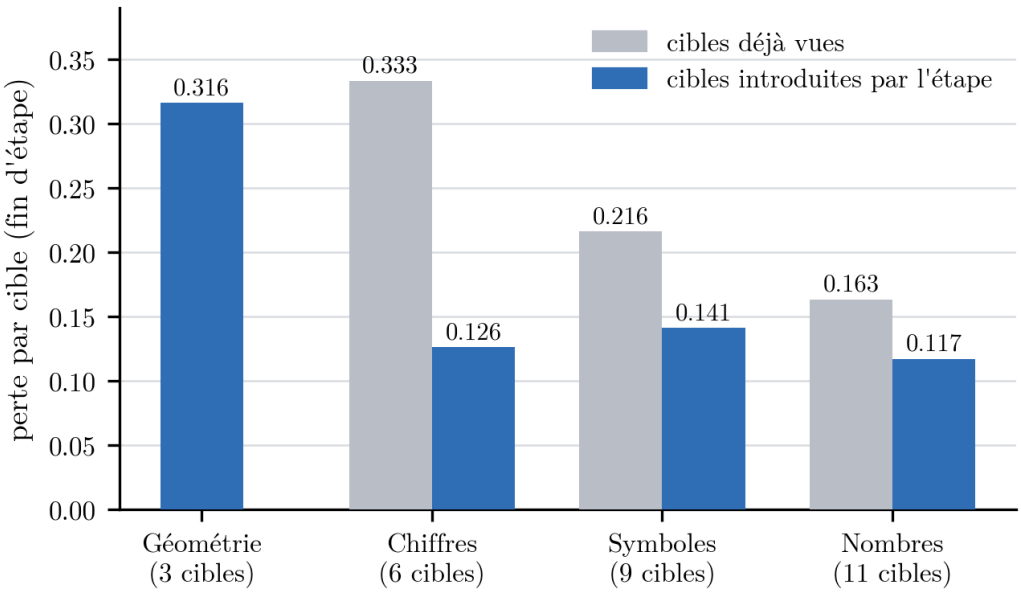


FIGURE 4.9 – Lecture visuelle du tableau 4.5 : pertes moyennes à la fin de chaque étape du curriculum, séparées entre cibles déjà vues et cibles introduites par l'étape, avec le nombre de cibles de chaque étape.

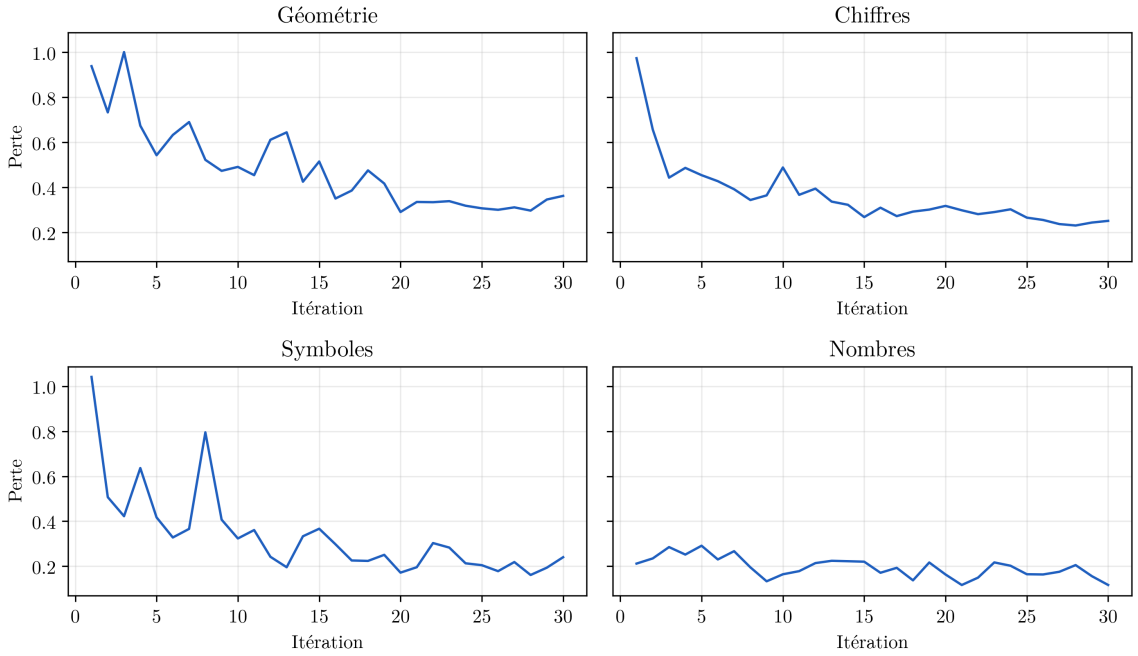


FIGURE 4.10 – Pertes enregistrées pendant les quatre étapes du curriculum réduit. Chaque graphe correspond à une étape ; l'axe des ordonnées est commun, ce qui permet de comparer l'amplitude des variations entre les étapes.

est attendue : l'introduction d'une cible modifie la distribution des épisodes du lot, donc le gradient, avant que l'optimisation ne se réajuste. Elle montre aussi que la perte rapportée en fin d'étape est une valeur ponctuelle d'une trajectoire bruitée, ce qui justifie de sauvegarder le meilleur état plutôt que le dernier.

Les visualisations montrent une couverture approximative des cibles symboliques. En l'état, les lettres et les nombres ne sont pas suffisamment lisibles pour soutenir une affirmation de reconnaissance ou de formation précise. L'amélioration de cette lisibilité exigera un protocole visuel et quantitatif dédié, davantage d'itérations, ainsi qu'une étude de l'effet du nombre de robots et de la densité de points de la forme.

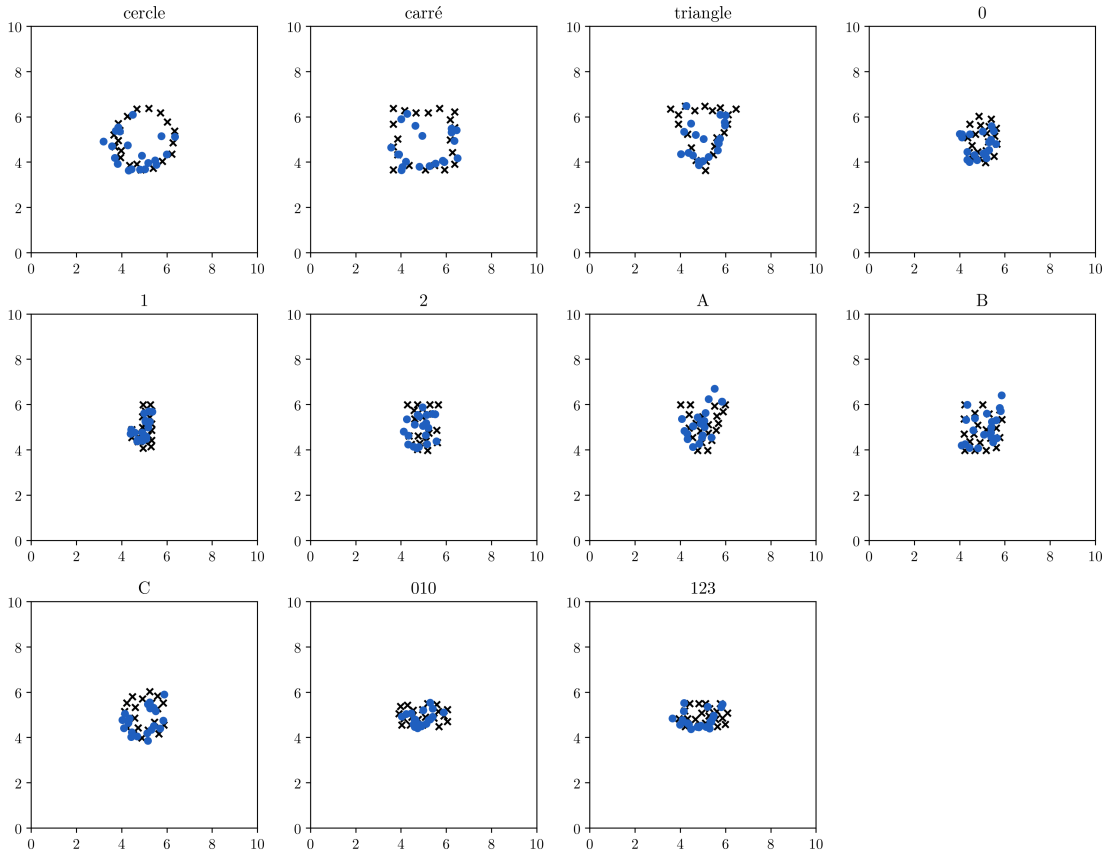


FIGURE 4.11 – Planche de visualisation issue de la dernière étape du curriculum réduit. Elle illustre des formations approximatives et ne constitue pas encore une validation de lisibilité des symboles.

4.8 Exemples, résultats négatifs et limites de lecture

Les exemples visuels ont une place importante dans ce projet parce que les métriques globales ne montrent pas tous les défauts d'une formation. Une configuration peut avoir une distance moyenne faible alors qu'un angle de lettre est mal dessiné ou que plusieurs robots forment un petit amas. C'est pourquoi les planches de visualisation présentent à

la fois les points de la cible et la position finale des robots. Elles permettent de repérer rapidement les zones non couvertes, les espacements irréguliers et les formes qui restent ambiguës.

Ces images ne doivent pas être sélectionnées uniquement parmi les meilleurs épisodes. Pour chaque campagne finale, il faudra conserver un ensemble fixe de graines, afficher au moins un exemple favorable et un exemple défavorable, puis relier ces cas aux statistiques. Cette procédure est particulièrement importante pour les symboles : une valeur moyenne ne peut pas établir à elle seule qu'une lettre est lisible par une personne.

Le curriculum réduit donne donc un résultat de faisabilité. Il montre que l'entraînement peut être poursuivi lorsqu'on ajoute de nouvelles cibles. Il ne suffit pas à dire que le modèle a appris un alphabet ou qu'il généralise à toutes les combinaisons de chiffres. Les lettres A, B et C ainsi que les nombres 010 et 123 sont les seules cibles symboliques de cette campagne. Les autres formes générées par le projet font partie du jeu de données, mais ne sont pas automatiquement des formes connues du modèle final.

4.9 Discussion des résultats

Le premier résultat est la validation de la chaîne expérimentale : le simulateur, le contrôleur à messages, la fonction de perte et l'optimisation permettent d'obtenir une formation circulaire plus proche de la cible que l'état initial. Le partage des paramètres permet d'appliquer la même politique à chaque robot, tandis que l'agrégation locale accepte un nombre variable de voisins. Le protocole pilote indique aussi que le GNN est plus proche du cercle que la règle de regroupement utilisée comme contrôle.

Cette observation doit rester limitée à son cadre. La règle de regroupement ne reçoit pas la même consigne de forme que le GNN. Elle répond donc à un problème différent et ne peut pas servir de preuve qu'un GNN est meilleur que toute règle décentralisée. Une comparaison plus juste devra inclure une règle classique ayant accès à la même cible, ainsi qu'une version apprise sans échange de messages.

Les résultats sur les pannes sont également préliminaires. La distance moyenne de Chamfer augmente peu entre les scénarios sans panne et les scénarios avec retrait de robots, mais le taux de réussite dépend du seuil choisi et tombe à zéro dans certaines conditions. Il serait donc imprudent de conclure à une robustesse générale. Pour répondre correctement à cette question, il faut multiplier les graines, calculer des intervalles de confiance et étudier les cas où l'essaim se fragmente ou couvre mal une partie de la cible.

Les limites de simulation doivent aussi être rappelées. Les robots possèdent un repère commun, les positions sont exactes et les messages ne subissent ni délai ni perte. La collision est pénalisée par une fonction continue plutôt que par un modèle de contact. Enfin, les lettres et les nombres issus du curriculum réduit restent des formations approximatives : leur présence dans une planche ne démontre ni une reconnaissance fiable ni une

généralisation à des symboles non vus.

La suite expérimentale doit donc consolider le protocole plutôt que l'élargir trop vite. Les priorités sont cinquante répétitions ou plus par condition, une baseline sans communication, des contrôleurs classiques recevant une consigne équivalente, du bruit de mesure et une réduction progressive de la portée de communication. Ces expériences permettront de distinguer ce qui relève réellement de l'échange de messages appris de ce qui provient de la consigne géométrique fournie au système.

4.10 Lien entre agrégation et résultats observés

L'agrégation moyenne rend le contrôleur compatible avec des voisinages de taille variable. Cette propriété est particulièrement pertinente dans les expériences de passage à l'échelle : lorsque le nombre de robots augmente, les robots situés au centre de l'arène peuvent recevoir plus de messages que ceux situés près du bord. Une somme brute de messages ferait augmenter la norme de l'entrée agrégée avec cette densité. La moyenne évite ce changement d'échelle immédiat, mais elle ne garantit pas à elle seule la qualité de formation.

Les résultats à 40 et 100 robots montrent que l'architecture ne s'effondre pas lorsque le nombre de nœuds change. Cette observation est cohérente avec le partage des paramètres et la moyenne des messages. Elle doit néanmoins être interprétée avec prudence : le succès accru à 100 robots peut aussi s'expliquer par la densité plus importante de points disponibles sur un même contour. Une ablation comparant moyenne, somme normalisée et absence de messages serait nécessaire pour attribuer précisément l'amélioration à la stratégie d'agrégation.

Le rapport avec le flocking est également limité mais instructif. La règle manuelle de regroupement utilise une interaction locale simple et produit une cohésion sans représenter explicitement le cercle. Le GNN reçoit une consigne supplémentaire et apprend ses réponses à partir d'exemples simulés. La comparaison actuelle montre donc la différence entre un comportement de rassemblement et un comportement conditionné par une cible, pas une opposition définitive entre méthodes manuelles et méthodes apprises.

4.11 Analyse des métriques

La distance de Chamfer est la métrique principale car elle répond directement à la question de la proximité entre la formation et la cible. Une valeur faible signifie que, en moyenne, les robots sont proches d'un point du contour et que les points du contour sont proches d'un robot. Elle ne décrit cependant pas la dynamique. Deux épisodes peuvent produire la même distance finale alors que l'un atteint la cible rapidement et que l'autre

oscille jusqu'au dernier pas. Le temps de convergence est donc une mesure complémentaire prévue pour les expériences finales.

Le taux de collision doit également être interprété avec soin. Dans cette étude, il est calculé à partir d'un seuil de sécurité et des positions simulées. Il ne mesure pas une collision mécanique avec contact, car le simulateur ne comporte pas de modèle de masse, de friction ou de rebond. Son rôle est de signaler les trajectoires qui placent trop souvent deux robots à une distance incompatible avec une marge de sécurité définie. Une baisse du taux de collision est favorable, mais elle ne compense pas une formation qui manquerait largement la cible.

La régularité des distances au plus proche voisin complète les deux mesures précédentes. Une formation peut être proche de la cible tout en laissant une partie du contour très dense et une autre presque vide. Cette métrique détecte une partie de ce phénomène, sans fournir une mesure parfaite de couverture. Dans la campagne finale, il serait utile de présenter les métriques ensemble dans un tableau par condition, plutôt que de sélectionner celle qui donne le résultat le plus favorable.

4.12 Menaces à la validité

La première menace concerne le nombre limité de répétitions. Dix épisodes par condition permettent de vérifier que les scripts fonctionnent et de détecter un écart important entre deux méthodes. Ils ne suffisent pas pour estimer précisément une moyenne, comparer des distributions ou établir un intervalle de confiance robuste. La cible de cinquante répétitions par condition est retenue pour la suite parce qu'elle rendra les agrégats plus informatifs, même si elle ne remplace pas une analyse statistique complète.

La deuxième menace est l'écart entre le simulateur et un système réel. Les robots du projet connaissent un repère commun, échangent des messages exacts et sont modélisés par une vitesse directement appliquée. Dans un robot physique, les estimations de position sont bruitées, les roues ont une inertie et les communications peuvent être retardées. Ces hypothèses rendent l'étude actuelle plus contrôlable, mais limitent la portée des conclusions.

La troisième menace concerne les formes elles-mêmes. Les nombres et lettres du curriculum réduit sont peu nombreux, et ils sont utilisés pendant l'entraînement. La capacité à former A, B ou C ne prouve pas une généralisation à toutes les lettres. Une expérience de généralisation devra réserver des formes complètes avant l'entraînement, puis mesurer la performance sur ces cibles non vues sans modifier les poids du modèle.

4.13 Plan d'expériences pour la campagne finale

La campagne finale doit être organisée autour de questions simples et mesurables. La première question concerne la qualité de formation : pour chaque catégorie de formes, quelle est la distance à la cible, quelle est la régularité de la répartition et quelle proportion d'épisodes franchit le seuil de succès ? La deuxième question concerne l'apport de la communication : que se passe-t-il lorsqu'on retire les messages de voisins en conservant la même consigne de forme ? La troisième question concerne la robustesse : comment évoluent les mesures lorsque des robots disparaissent, lorsque le rayon de communication diminue ou lorsque les positions sont bruitées ?

Chaque condition doit être fixée avant de consulter les résultats. Par exemple, une expérience de panne peut combiner trois tailles d'essaim, quatre fractions de retrait et cinquante graines. Les mêmes positions initiales peuvent être réutilisées pour comparer deux contrôleurs, afin que l'écart observé ne soit pas causé par une série de départs plus faciles pour l'une des méthodes. Les données brutes doivent ensuite être conservées pour permettre le recalcul des moyennes, écarts-types et graphiques.

Une présentation complète devra associer un tableau numérique à une figure. Le tableau donne les valeurs précises et leur dispersion. La figure montre la tendance quand un facteur varie, par exemple le pourcentage de pannes. Les images finales et les animations jouent un troisième rôle : elles permettent de comprendre la nature spatiale d'un échec. Cette complémentarité évite de réduire un comportement collectif à un seul nombre.

4.14 Interprétation prudente des résultats pilotes

Le pilote est utile parce qu'il valide l'enchaînement complet des scripts. Il produit des épisodes, agrège les résultats et génère des graphiques sans intervention manuelle. Il met aussi en évidence un écart important entre le GNN conditionné par la cible et la règle de regroupement. Cet écart justifie de poursuivre les expériences, mais il ne suffit pas pour une conclusion forte sur la supériorité d'une méthode.

La première raison est l'asymétrie d'information déjà discutée. La seconde est le nombre de répétitions. La troisième est que les résultats concernent principalement une cible circulaire. Une méthode peut être très performante pour un contour régulier et moins convaincante pour une lettre présentant des angles ou des traits éloignés. L'objectif de la campagne finale est donc de remplacer une affirmation générale par un ensemble de résultats précis, chacun associé à une forme, une taille d'essaim et une perturbation donnée.

Cette prudence améliore la valeur du mémoire. Un résultat négatif, comme une lettre difficile à couvrir ou un effondrement lorsque la portée devient trop faible, n'est pas un échec de rédaction. Il renseigne sur les limites de la politique et aide à déterminer quelles

observations ou quels mécanismes doivent être ajoutés. Dans un travail sur les systèmes distribués, expliquer pourquoi une méthode cesse de fonctionner est souvent aussi utile que montrer ses meilleurs exemples.

Chapitre 5

Discussion et conclusion

5.1 Interprétation des résultats

Le premier résultat utile est la faisabilité de la chaîne complète : le simulateur, le contrôleur à messages, la fonction de perte et l'optimisation convergent vers une formation circulaire plus proche de la cible que l'état initial. Les paramètres partagés permettent par construction d'appliquer le même réseau à chaque robot, tandis que l'agrégation locale permet de gérer un nombre variable de voisins. Le protocole pilote indique également une meilleure adéquation au cercle que la règle de regroupement implémentée.

Cette observation doit rester limitée à sa portée. La baseline de regroupement n'utilise pas la même information de consigne que le GNN ; elle n'est donc pas une baseline apprise ni une baseline classique ajustée à la tâche de cercle. L'écart de distance de Chamfer ne répond pas encore à la question « le GNN est-il meilleur à information égale ? ». Il répond seulement à une question plus modeste : dans la configuration pilotée, la politique entraînée se rapproche davantage du cercle qu'une règle qui vise le regroupement.

Un point de protocole doit être précisé ici, car il conditionne l'interprétation. Le pilote évalue le modèle issu du dernier niveau du curriculum réduit, et non le modèle entraîné sur le seul cercle. Cette décision a été prise pour évaluer le contrôleur réellement destiné à la démonstration, mais elle introduit une variable supplémentaire : les résultats du pilote reflètent l'effet conjoint du curriculum et de l'architecture. Une campagne définitive devra séparer les deux, en évaluant à la fois le modèle spécialisé sur le cercle et le modèle issu du curriculum, sur les mêmes épisodes.

La dégradation limitée de la distance moyenne après retrait de 20 % des robots constitue un signal à étudier, et non une preuve de robustesse générale. Le taux de succès, qui devient nul à ce niveau de pannes selon le seuil retenu, rappelle que la moyenne de Chamfer seule ne caractérise pas toute la performance. Le pas de convergence, qui n'est presque jamais atteint à vingt robots, montre qu'une distance finale acceptable peut coexister avec une trajectoire qui n'a pas stabilisé la formation dans la durée allouée.

5.2 Comparaison avec les travaux antérieurs

Il est instructif de situer les résultats par rapport aux travaux cités au chapitre 2, en distinguant ce qui relève de l'architecture de ce qui relève du protocole.

Tolstaya et al. montrent qu'un contrôleur GNN peut être entraîné pour des tâches de robotique en essaim sur un graphe de communication mobile [31]. Le présent travail s'inscrit dans la même famille d'architectures, mais la tâche diffère : il s'agit ici de converger vers une géométrie de cible donnée, non de suivre une trajectoire ou d'éviter des obstacles. L'objectif de forme impose une couverture du contour, qui n'a pas d'équivalent direct dans une tâche de navigation.

Gama et al. étudient les GNN comme contrôleurs décentralisés et explorent le transfert vers des équipes plus grandes ou des scénarios nouveaux, notamment par imitation [7, 8]. Le passage à l'échelle observé dans le pilote est cohérent avec leurs conclusions : l'architecture ne s'effondre pas lorsque le nombre de robots change. La différence tient au mode d'apprentissage : ici, le signal vient d'une perte de forme évaluée en fin d'épisode et remontée par simulation différentiable, plutôt que d'une imitation d'expert. Ce choix évite la dépendance à un contrôleur de référence, au prix d'un horizon de rétropropagation égal à la durée de l'épisode.

Jiang, Huang et Guo proposent une formation décentralisée de bout en bout fondée sur un GNN [15]. Le rapprochement est direct, mais les hypothèses de perception diffèrent : ces travaux s'appuient sur des observations issues de capteurs embarqués transformées en cartes locales, alors que le prototype travaille avec des états simulés et une consigne géométrique fournie avant l'épisode. Cette différence doit être gardée à l'esprit : la comparaison des performances n'a de sens qu'à hypothèses de perception comparables.

Les approches à contrôleurs écrits à la main pour la génération de formes [12, 18] restent la référence à atteindre. Leur force est l'interprétabilité et l'absence d'entraînement ; leur limite est la conception spécifique à chaque famille de formes. Le mémoire ne prétend pas les remplacer. Il montre qu'une loi apprise, exécutée localement, peut être entraînée de façon reproductible sur un ensemble de formes hétérogènes. La comparaison équitable — une règle manuelle recevant une consigne de contour comparable — n'a pas encore été réalisée et constitue la principale dette expérimentale du travail.

5.3 Menaces à la validité

5.3.1 Validité interne

Le principal facteur limitant est le nombre de répétitions. Dix épisodes par condition permettent de valider une chaîne de mesure et de détecter des écarts importants ; ils ne permettent pas d'estimer précisément une moyenne, encore moins de comparer des distri-

butions. Un facteur secondaire mais réel est le couplage entre architecture et protocole : le modèle évalué dans le pilote résulte d'un curriculum, ce qui brouille l'attribution des résultats au seul passage de messages.

L'absence d'une baseline apprise sans communication constitue le deuxième facteur. Tant qu'elle n'existe pas, l'écart mesuré entre le GNN et la règle de regroupement reste un écart entre un système conditionné par une cible et un système qui l'ignore. Enfin, la graine unique d'entraînement impose de considérer les résultats comme le comportement d'un entraînement particulier, non comme la performance attendue de l'architecture.

5.3.2 Validité externe

Les robots du simulateur disposent d'un repère commun, de positions exactes et de communications sans délai ni perte. Les collisions sont pénalisées par une fonction continue plutôt que résolues par une dynamique de contact. Un robot en panne disparaît proprement du graphe, alors qu'un défaillant réel peut rester immobile, devenir un obstacle ou transmettre des valeurs aberrantes. L'arène est finie et ses limites modifient les trajectoires, de sorte qu'aucun résultat ne peut être extrapolé à un espace ouvert. Les épisodes sont par ailleurs courts : vingt pas dans le pilote, ce qui correspond à une fenêtre bien plus réduite qu'une mission réelle.

Ces hypothèses rendent l'étude contrôlable. Elles limitent également la portée des conclusions : les résultats établissent une faisabilité en simulation 2D, non un transfert vers des plateformes physiques.

5.3.3 Validité de construit

La construction même des mesures comporte des approximations. La distance de Chamfer est sensible aux deux ensembles mais aveugle à la lisibilité d'un symbole : deux configurations peuvent avoir une distance voisine alors que l'une reste interprétable et l'autre non. Le seuil de succès, fixé à 0,5 dans le pilote, est un choix opérationnel dont dépend directement le pourcentage de réussite. Le taux de collision mesure la fréquence de rapprochements sous une distance de sécurité, non des impacts mécaniques. La régularité des voisins détecte des amas, sans garantir une couverture complète du contour.

Ces limites ne rendent pas les mesures inutiles ; elles imposent de les présenter ensemble, avec leur définition, plutôt que de sélectionner celle qui produit le résultat le plus favorable.

5.4 Implications pour la conception de contrôleurs décentralisés

Trois enseignements de conception se dégagent du travail, sous réserve des réserves précédentes.

Premièrement, la combinaison consigne partagée et messages locaux est suffisante, en simulation, pour produire des formations non triviales avec un seul jeu de paramètres. Cela ne signifie pas que la consigne soit anodine : les expériences du chapitre 4 montrent qu'elle porte l'essentiel de l'information de géométrie, et que le rôle du passage de messages reste à isoler par une ablation sans communication.

Deuxièmement, l'agrégation par moyenne fournit une échelle stable lorsque la densité change. Les résultats à 40 et 100 robots n'indiquent aucune dégradation brutale, ce qui est cohérent avec la normalisation par le nombre de voisins. Une somme non normalisée, ou une agrégation par attention non régularisée, n'offre pas cette propriété et devrait être évaluée avant tout passage à une taille d'essaim beaucoup plus grande.

Troisièmement, la progression de difficulté est utile mais ne résout pas l'oubli. Conserver les cibles des niveaux précédents réduit le risque de perte des acquis ; cela ne dispense pas de mesurer, à chaque niveau, la performance sur l'ensemble des formes déjà vues. Le tableau des pertes par cible du chapitre 4 est un premier pas dans cette direction.

5.5 Considérations pratiques et sociales

Les essaims robotiques sont souvent justifiés par leur faible coût unitaire et leur redondance : des plateformes comme les Kilobot illustrent la possibilité de déployer des centaines de robots simples [28]. Cette caractéristique transforme aussi les enjeux d'évaluation. Un contrôleur qui fonctionne sur simulateur mais dépend d'hypothèses de repère ou de communication fragiles peut imposer des coûts de recalibrage considérables à l'échelle du groupe.

Un second point concerne l'usage de la simulation. Entraîner et valider numériquement avant tout déploiement physique réduit le nombre d'essais sur matériel, donc les consommations et les pertes associées. La reproductibilité des campagnes, obtenue ici par configuration et graine explicites, participe de cette sobriété : rejouer une expérience coûte du temps processeur, régénérer des robots coûte des matières.

Enfin, une formation de forme par essaim est un dispositif visible dans l'espace public. Toute extension vers des plateformes réelles devra traiter les questions de bruit, d'occupation de l'espace et de consentement des personnes concernées, ainsi que les usages ambivalents d'un tel dispositif. Ces considérations ne sont pas abordées expérimentalement dans ce mémoire ; elles figurent au titre des limites de périmètre.

5.6 Suite expérimentale

Contrôleur	Messages des voisins	Consigne de forme	Paramètres appris	Statut
Règle de regroupement (cohésion–répulsion)	●	aucune	○	pilote
GNN, 2 tours (modèle du curriculum)	●	identifiant	●	pilote
Règle manuelle avec consigne de forme	●	vecteur contour	○	à faire
Réseau appris sans communication	○	identifiant	●	à faire
GNN à 1 tour et à 3 tours	●	identifiant	●	à faire

disque plein : oui ; disque vide : non — comparaisons à information équivalente prévues au chapitre 4 (§4.13) et au §5.6

FIGURE 5.1 – Contrôleurs comparés ou prévus : présence des messages de voisins, de la consigne de forme et de paramètres appris, ainsi que le statut de chaque comparaison. Seules les deux premières lignes ont été évaluées dans le pilote ; les autres constituent les comparaisons à information équivalente à réaliser.

Les prochaines expériences doivent inclure au moins 50 départs par condition, des intervalles de confiance, du bruit de position et de communication, ainsi qu'une réduction systématique de la portée radio. Une politique apprise sans passage de messages, recevant exactement la même consigne de forme, constitue la baseline prioritaire. Des contrôleurs classiques équipés de la même consigne peuvent compléter cette comparaison. Il faudra également séparer les mesures de couverture de forme, de régularité de répartition et de sécurité, puis définir des critères de lisibilité adaptés aux symboles. La figure 5.1 récapitule, contrôleur par contrôleur, l'information disponible et le statut de chaque comparaison.

Un axe complémentaire concerne le raffinement de l'agrégation : comparer la moyenne à une agrégation tronquée ou pondérée permettrait de mesurer la sensibilité du contrôleur à un voisin défaillant. Un axe symétrique concerne la profondeur : tester un, deux et trois tours de messages à budget d'entraînement identique permettrait d'attribuer les gains observés à la portée informationnelle plutôt qu'à la capacité du modèle.

À plus long terme, le repère commun devrait être remplacé par des observations relatives, une localisation distribuée ou une balise mesurable. Le passage d'une dynamique à vitesse directe à un modèle cinématique ou dynamique plus réaliste permettra ensuite d'évaluer la transférabilité vers des robots physiques.

5.7 Conclusion et perspectives

Ce travail a établi une chaîne expérimentale reproductible pour étudier la formation de formes par un essaim contrôlé par GNN. Le système associe un simulateur 2D, des formes représentées par nuages de points, une politique locale à paramètres partagés et un apprentissage différentiable. Le cercle constitue une première validation visuelle et quantitative du fonctionnement de cette chaîne. Les modules de génération de données, d'entraînement, d'évaluation et de visualisation sont séparés afin que les expériences puissent être rejouées à partir de leurs configurations.

Le résultat principal n'est pas encore une démonstration de déploiement sur robots physiques. Il est plus précis : dans le simulateur retenu, une politique GNN partagée peut recevoir des informations locales, échanger des messages avec ses voisins et produire des mouvements qui rapprochent l'essaim d'une cible. Le modèle se laisse également exécuter avec un nombre de robots différent de celui utilisé pendant l'entraînement. Cette propriété découle du partage des paramètres et de l'agrégation des messages, mais sa qualité doit rester vérifiée expérimentalement pour chaque scénario.

Le protocole pilote suggère que le contrôleur appris atteint une meilleure adéquation à une cible circulaire qu'une règle locale de regroupement, y compris dans les scénarios de retrait étudiés. Cette conclusion est volontairement restreinte : le pilote ne comporte que dix graines par condition et la baseline ne dispose pas de la même consigne. Les formes symboliques sont, elles aussi, encore au stade de couverture approximative. Elles montrent que le pipeline traite plusieurs types de contours ; elles ne permettent pas encore d'affirmer une formation lisible et généralisable de lettres ou de nombres.

Les premières perspectives sont expérimentales. Il faut augmenter le nombre de répétitions, fixer un jeu de formes non vues, mesurer des intervalles de confiance et conserver les épisodes favorables comme les épisodes défavorables. Il faut aussi ajouter des baselines équitables : une politique sans communication, une règle manuelle recevant une consigne comparable et plusieurs profondeurs de passage de messages. Ces comparaisons permettront de mesurer l'apport spécifique de l'agrégation locale plutôt que l'effet général de l'information de cible.

Les perspectives techniques concernent le réalisme du simulateur. Des bruits de position et de communication, des délais, des obstacles et une dynamique de robot non holonome rapprocheraient le cadre de situations réelles. À plus long terme, la consigne géométrique pourrait être fournie par une perception locale, une balise ou une carte distribuée. Le passage sur une plateforme physique demanderait alors une estimation de pose, une stratégie de sécurité et une validation progressive, d'abord sur quelques robots puis sur un essaim plus grand.

Le projet répond donc partiellement à la question initiale. Il montre la faisabilité d'un contrôleur GNN local pour une formation de formes en simulation et fournit les

outils nécessaires à une évaluation plus rigoureuse. La suite du travail devra déterminer, avec des expériences mieux contrôlées, dans quelles conditions cette approche apporte un avantage réel par rapport aux méthodes de contrôle distribué classiques.

Bibliographie

[1] Filipe de Avila Belbute-Peres et al. “End-to-end differentiable physics for learning and control”. In : *Advances in Neural Information Processing Systems*. T. 31. 2018.

[2] Peter W. Battaglia et al. “Relational Inductive Biases, Deep Learning, and Graph Networks”. In : *arXiv preprint arXiv :1806.01261* (2018).

[3] Manuele Brambilla et al. “Swarm robotics : a review from the swarm engineering perspective”. In : *Swarm Intelligence* 7 (2013), p. 1-41. doi : 10.1007/s11721-012-0075-2.

[4] Scott Camazine et al. *Self-organization in biological systems*. Princeton University Press, 2001.

[5] Leon O. Chua et Yang Yang. “Cellular neural networks : Theory”. In : *IEEE Transactions on Circuits and Systems* 35.10 (1988), p. 1257-1272. doi : 10.1109/31.7600.

[6] Felipe Cucker et Steve Smale. “Emergent behavior in flocks”. In : *IEEE Transactions on Automatic Control* 52.5 (2007), p. 852-862. doi : 10.1109/TAC.2007.895842.

[7] Fernando Gama, Ekaterina Tolstaya et Alejandro Ribeiro. “Graph Neural Networks for Decentralized Controllers”. In : *arXiv preprint arXiv :2003.10280* (2020).

[8] Fernando Gama et al. “Synthesizing Decentralized Controllers with Graph Neural Networks and Imitation Learning”. In : *IEEE Transactions on Signal Processing* 70 (2022), p. 202-215. doi : 10.1109/TSP.2021.3139935.

[9] Justin Gilmer et al. “Neural Message Passing for Quantum Chemistry”. In : *ICML*. 2017, p. 1263-1272.

[10] Ian J. Goodfellow et al. “An empirical investigation of catastrophic forgetting in gradient-based neural networks”. In : *arXiv preprint arXiv :1312.6211* (2013).

[11] William L. Hamilton, Rex Ying et Jure Leskovec. “Inductive Representation Learning on Large Graphs”. In : *Advances in Neural Information Processing Systems*. T. 30. 2017. url : https://arxiv.org/abs/1706.02216.

[12] M. Ani Hsieh, Vijay Kumar et Luiz Chaimowicz. “Decentralized controllers for shape generation with robotic swarms”. In : *Robotica* 26.5 (2008), p. 691-701. doi : 10.1017/S0263574708004323.

[13] Yuanming Hu et al. “DiffTaichi : differentiable programming for physical simulation”. In : *International Conference on Learning Representations (ICLR)*. 2020.

[14] Ali Jadbabaie, Jie Lin et A. Stephen Morse. “Coordination of groups of mobile autonomous agents using nearest neighbor rules”. In : *IEEE Transactions on Automatic Control* 48.6 (2003), p. 988-1001. doi : 10.1109/TAC.2003.812781.

[15] Chao Jiang, Xinchi Huang et Yi Guo. “End-to-end decentralized formation control using a graph neural network-based learning method”. In : *Frontiers in Robotics and AI* 10 (2023). doi : 10.3389/frobt.2023.1285412.

[16] Thomas N. Kipf et Max Welling. “Semi-supervised classification with graph convolutional networks”. In : *International Conference on Learning Representations (ICLR)*. 2017.

[17] James Kirkpatrick et al. “Overcoming catastrophic forgetting in neural networks”. In : *Proceedings of the National Academy of Sciences* 114.13 (2017), p. 3521-3526. doi : 10.1073/pnas.1611835114.

[18] Guannan Li et al. “Decentralized progressive shape formation with robot swarms”. In : *Autonomous Robots* 43.6 (2019), p. 1505-1521. doi : 10.1007/s10514-018-9807-5.

[19] Ryan Lowe et al. “Multi-agent actor-critic for mixed cooperative-competitive environments”. In : *Advances in Neural Information Processing Systems*. T. 30. 2017, p. 6379-6388.

[20] Mehran Mesbahi et Magnus Egerstedt. *Graph theoretic methods in multiagent networks*. Princeton University Press, 2010.

[21] Reza Olfati-Saber. “Flocking for Multi-Agent Dynamic Systems : Algorithms and Theory”. In : *IEEE Transactions on Automatic Control* 51.3 (2006), p. 401-420. doi : 10.1109/TAC.2005.864190.

[22] Reza Olfati-Saber et Richard M. Murray. “Consensus problems in networks of agents with switching topology and time-delays”. In : *IEEE Transactions on Automatic Control* 49.9 (2004), p. 1520-1533. doi : 10.1109/TAC.2004.834113.

[23] Dean A. Pomerleau. “ALVINN : an autonomous land vehicle in a neural network”. In : *Advances in Neural Information Processing Systems*. T. 1. 1989, p. 305-313.

[24] Tabish Rashid et al. “QMIX : monotonic value function factorisation for deep multi-agent reinforcement learning”. In : *Proceedings of the 35th International Conference on Machine Learning*. T. 80. Proceedings of Machine Learning Research. PMLR, 2018, p. 4295-4304.

[25] Wei Ren et Randal W. Beard. “Consensus seeking in multiagent systems under dynamically changing interaction topologies”. In : *IEEE Transactions on Automatic Control* 50.5 (2005), p. 655-661. doi : 10.1109/TAC.2005.846556.

[26] Craig W. Reynolds. “Flocks, Herds, and Schools : A Distributed Behavioral Model”. In : *SIGGRAPH*. 1987, p. 25-34. doi : 10.1145/37401.37406.

[27] Stéphane Ross, Geoffrey Gordon et Drew Bagnell. “A reduction of imitation learning and structured prediction to no-regret online learning”. In : *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics*. T. 15. Proceedings of Machine Learning Research. PMLR, 2011, p. 627-635.

[28] Michael Rubenstein et al. “Kilobot : a low cost robot with scalable operations designed for collective behaviors”. In : *Robotics and Autonomous Systems* 62.7 (2014), p. 966-975. doi : 10.1016/j.robot.2013.08.006.

[29] Erol Şahin. “Swarm robotics : from sources of inspiration to domains of application”. In : *Swarm Robotics, SAB 2004 International Workshop*. T. 3342. Lecture Notes in Computer Science. Springer, 2005, p. 10-20. doi : 10.1007/978-3-540-30552-1_2.

[30] Franco Scarselli et al. “The graph neural network model”. In : *IEEE Transactions on Neural Networks* 20.1 (2009), p. 61-80. doi : 10.1109/TNN.2008.2005605.

[31] Ekaterina Tolstaya et al. “Learning Decentralized Controllers for Robot Swarms with Graph Neural Networks”. In : *CoRL*. 2020, p. 671-682.

[32] Petar Veličković et al. “Graph attention networks”. In : *International Conference on Learning Representations (ICLR)*. 2018.

[33] Tamás Vicsek et al. “Novel type of phase transition in a system of self-driven particles”. In : *Physical Review Letters* 75.6 (1995), p. 1226-1229. doi : 10.1103/PhysRevLett.75.1226.

[34] Zonghan Wu et al. “A comprehensive survey on graph neural networks”. In : *IEEE Transactions on Neural Networks and Learning Systems* 32.1 (2021), p. 4-24. doi : 10.1109/TNNLS.2020.2978386.

[35] Manzil Zaheer et al. “Deep sets”. In : *Advances in Neural Information Processing Systems*. T. 30. 2017, p. 3391-3401.